

 **Islington college**
(इस्लिङ्टन कलेज)

CS4051NI/CC4059NI Fundamentals of Computing

70% Individual Coursework

2024/25 Spring

Student Name: Suprem Thapa London

Met ID: 24046523

College ID: NP01NT4A240123

Assignment Due Date: Wednesday , May 14, 2025

Assignment Submission Date: Wednesday , May 14, 2025

Word Count: 6075

13% Overall Similarity

The combined total of all matches, including overlapping sources, for each database.

Match Groups

- 68 Not Cited or Quoted 11%
Matches with neither in-text citation nor quotation marks
- 0 Missing Quotations 0%
Matches that are still very similar to source material
- 13 Missing Citation 2%
Matches that have quotation marks, but no in-text citation
- 0 Cited and Quoted 0%
Matches with in-text citation present, but no quotation marks

Top Sources

- 5% Internet sources
- 3% Publications
- 10% Submitted works (Student Papers)

Integrity Flags

0 Integrity Flags for Review

Our system's algorithms look deeply at a document for any inconsistencies that would set it apart from a normal submission. If we notice something strange, we flag it for you to review.

A flag is not necessarily an indicator of a problem. However, we'd recommend you focus your attention there for further review.

Match Groups

- 68 Not Cited or Quoted 11%
Matches with neither in-text citation nor quotation marks
- 0 Missing Quotations 0%
Matches that are still very similar to source material
- 13 Missing Citation 2%
Matches that have quotation marks, but no in-text citation
- 0 Cited and Quoted 0%
Matches with in-text citation present, but no quotation marks

Top Sources

- 5% Internet sources
- 3% Publications
- 10% Submitted works (Student Papers)

Top Sources

The sources with the highest number of matches within the submission. Overlapping sources will not be displayed.

1	Internet	
	www.coursehero.com	3%
2	Submitted works	
	islingtoncollege on 2025-05-12	1%
3	Submitted works	
	Henley College Coventry, Coventry on 2025-05-08	<1%
4	Submitted works	
	University of Dayton on 2023-11-13	<1%
5	Submitted works	
	Bromley College on 2018-04-22	<1%
6	Submitted works	
	islingtoncollege on 2025-05-14	<1%
7	Submitted works	
	Asia Pacific University College of Technology and Innovation (UCTI) on 2024-06-14	<1%
8	Internet	
	base32encode.com	<1%
9	Submitted works	
	islingtoncollege on 2025-01-22	<1%
10	Submitted works	
	University of Bolton on 2024-11-08	<1%

Table of Contents

1	INTRODUCTION	1
1.1	AIM AND OBJECTIVES	2
1.	Aim:	2
1.1.1	Objectives:	2
1.2	Tools and Technology Used	3
2	Discussion and Analysis	5
2.1	FLOWCHART	7
2.2	Pseudocode for read.py:	10
2.3	Pseudocode for write.py:	14
2.4	Pseudocode for operations.py	23
2.5	Pseudocode for main.py	32
2.6	DATA STRUCTURE	37
3	PROGRAM	39
4	TESTING	50
	<i>Show the purchase and sale of the product (Show the complete process)</i>	50
4.1	TEST 1	58
4.2	TEST 2	59
4.3	TEST 3	61
4.4	TEST 4	65
4.5	TEST 5	69
5	CONCLUSION	75
6	APPENDIX	76

TABLE OF FIGURE

Figure 1 Python	3
Figure 2 IDLE	4
Figure 3 DRAW>IO	4
Figure 4 FLOWCHART START	7
Figure 5 Flowchart for B	8

Figure 6 Flowchart for C and END	9
Figure 7 READ.PY EXAMPLE	37
Figure 8 Code of READ.PY	40
Figure 9 Code of WRITE.PY	41
Figure 10	42
Figure 11	43
Figure 12 Code of OPERATIONS.PY	44
Figure 13	45
Figure 14	46
Figure 15	47
Figure 16 Code of MAIN.PY	48
Figure 17	49
Figure 18 Showing the running process of my code	50
Figure 19 invoice of sale	51
Figure 20	Error! Bookmark not defined.
Figure 21 Selecting option 2	52
Figure 22	Error! Bookmark not defined.
Figure 23 Invoice of restock product	53
Figure 24 Runing option 3 Figure 25 Running option 4	54
Figure 26 Txt file of product	54
Figure 27 Invoice after sale	55
Figure 28 Invoice in txt after restock	56
Figure 29 Running option 4	57
Figure 30 finding invalid. input	59
Figure 31 Provide the negative value as input, Provide the non existed value as input ..	61
Figure 32 Show complete purchase process and output in shell	63
Figure 33 Finally show the purchased products details in a text file	64
Figure 34 Show the complete sales process of the product(s) Show output in the shell as well.	67
Figure 35 Finally show the sold product(s) details in text file	68

Figure 36 Show the update in stock of product(s)	70
Figure 37 Show the update in stock of quantity	71
Figure 38 Show the update in stock of quantity	72
Figure 39 Show the quantity being deducted while purchasing the product	73
Figure 40 Show the quantity being added while purchasing the product	73
Figure 41 Show the quantity being added while selling the product	74
Figure 42 Show the quantity being deducting while selling the product	74
TABLE OF CONTAIN	

Table 1	59
Table 2	60
Table 3	62
Table 4	66
Table 5	69

1 INTRODUCTION

In the dynamic and competitive world of retail and wholesale business, efficient management of inventory, sales, and stock replenishment is crucial. The Wholesale Management System developed as part of this Python coursework aims to streamline the core operations of a wholesale business, specifically focusing on product sales and stock management.

The system is designed to handle fundamental operations such as selling products to customers, purchasing stock from manufacturers, and maintaining an up-to-date product inventory. Implemented using the Python programming language, the system adheres to best practices, including modular coding, input validation, and error handling. It leverages multiple Python files to organize the code effectively: `read.py`, `write.py`, `main.py`, and `operations.py`, each responsible for specific functionalities.

Furthermore, the system features a user-friendly console interface that guides users through essential tasks while ensuring data integrity and accuracy. Error handling is implemented to manage invalid inputs, and proper naming conventions are maintained to enhance code readability and maintainability.

The primary objective of this report is to document the development process, explain the system's features, and present a comprehensive analysis of the solution's efficiency and usability. The report also highlights the challenges encountered during implementation and the strategies employed to overcome them.

This project not only demonstrates the application of core Python programming concepts but also provides a practical solution for managing a wholesale business efficiently.

1.1 AIM AND OBJECTIVES

1. Aim:

The aim of this project is to design and develop a Python-based Wholesale Management System for skincare products, which provides a user-friendly interface for managing product sales, restocking, and inventory tracking. The system is intended to simplify the management of wholesale operations while ensuring accurate and efficient product tracking.

1.1.1 Objectives:

1. Modular Design: Develop a modular program by dividing the code into four separate Python files (main.py, read.py, write.py, and operations.py), ensuring clear separation of responsibilities for better code management.
2. Product Management: Implement a system that allows the user to:
 - Sell skincare products to customers with automated stock adjustment.
 - Restock products by purchasing them from manufacturers.
 - Display a list of available products with their details (name, brand, quantity, price, and origin).
3. Error Handling: Integrate try-except blocks to manage user input errors, file reading errors, and other potential runtime exceptions without crashing the program.
4. File Handling: Use external file storage (Products.txt) to maintain product details, ensuring that data is persistent across program runs.
5. User-Friendly Interface: Design a clear, text-based user interface that allows users to easily navigate between selling, purchasing, and exiting the system.
6. Data Accuracy: Ensure that product quantities are accurately updated after each transaction, maintaining the integrity of stock data.

7. Scalability: Design the system in a way that allows for future enhancements, such as adding new products, modifying prices, and introducing additional user options.

1.2 Tools and Technology Used

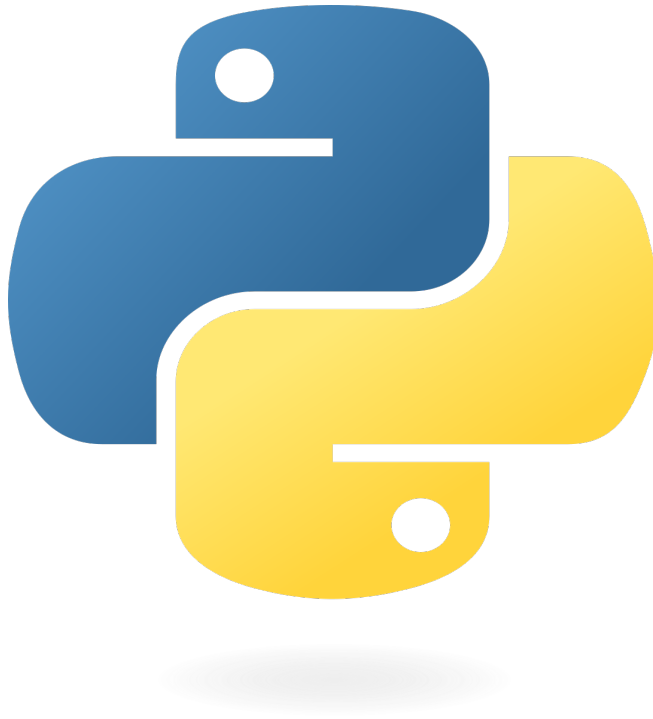


Figure 1 Python

1. Programming Language:

- Python: Python is a high-level, interpreted programming language known for its simplicity, readability, and wide range of applications. Developed by Guido van Rossum and first released in 1991, Python has become one of the most popular programming languages due to its beginner-friendly syntax and extensive standard library.

IDLE

IDLE (Integrated Development and Learning Environment) is the official Python IDE (Integrated Development Environment) that comes bundled with Python when installed.

SUPREM THAPA

Designed primarily for beginners, IDLE provides a simple, user-friendly environment for writing, testing, and debugging Python code.



Figure 2 IDLE

Draw.io

draw.io (now known as diagrams.net) is an open-source, free-to-use online diagramming tool that allows users to create a wide variety of diagrams, including flowcharts, network diagrams, UML diagrams, mind maps, and more. It is widely used for creating visual representations of processes, algorithms, and system architectures.



Figure 3 DRAW>IO

Terminal and Command Line Interface (CLI)

A **Terminal** or **Command Line Interface (CLI)** is a text-based interface used to interact with an operating system. Unlike graphical user interfaces (GUIs), which rely on visual elements like buttons and windows, a CLI allows users to execute commands directly by typing them.

2 Discussion and Analysis

Algorithem

Step 1: Start

Step 2: Display welcome message

Step 3: Load product data from file

Step 4: Display main menu options

Step 5: Input user choice (1-4)

Step 6: If user choice == 1 then goto 10 (Sell Products)

Step 7: Else if user choice == 2 then goto 18 (Purchase Products)

Step 8: Else if user choice == 3 then goto 31 (Display Products)

Step 9: Else if user choice == 4 then goto 34 (Exit)

Step 10: Input customer name

Step 11: Display available products

Step 12: Input product ID to sell (0 to finish)

Step 13: If product ID == 0 and items selected then goto 16

Step 14: If product ID invalid or out of stock then goto 12

Step 15: Input quantity, update stock, add to bill, goto 12

Step 16: Calculate bill with VAT

SUPREM THAPA

Step 17: Generate and save invoice, update files, goto 4

Step 18: Display purchase options (1: Restock, 2: Add new, 3: Finish)

Step 19: If choice == 1 then goto 20 (Restock)

Step 20: Else if choice == 2 then goto 24 (Add new)

Step 21: Else if choice == 3 then goto 28 (Finish purchase)

Step 22: Display products, input product ID to restock

Step 23: Input quantity, update stock, add to purchase list, goto 18

Step 24: Input new product details (name, brand, quantity, price, origin)

Step 25: Add new product to inventory, add to purchase list, goto 18

Step 26: If no items purchased then goto 4

Step 27: Calculate purchase with bill

Step 28: Generate and save purchase invoice, update files, goto 4

Step 29: Else print invalid choice, goto 18

Step 30: Else print invalid choice, goto 5

Step 31: Display all available products with details

Step 32: Wait for user to continue

Step 33: Goto 4

Step 34: Print exit message

Step 35: Stop

2.1 FLOWCHART

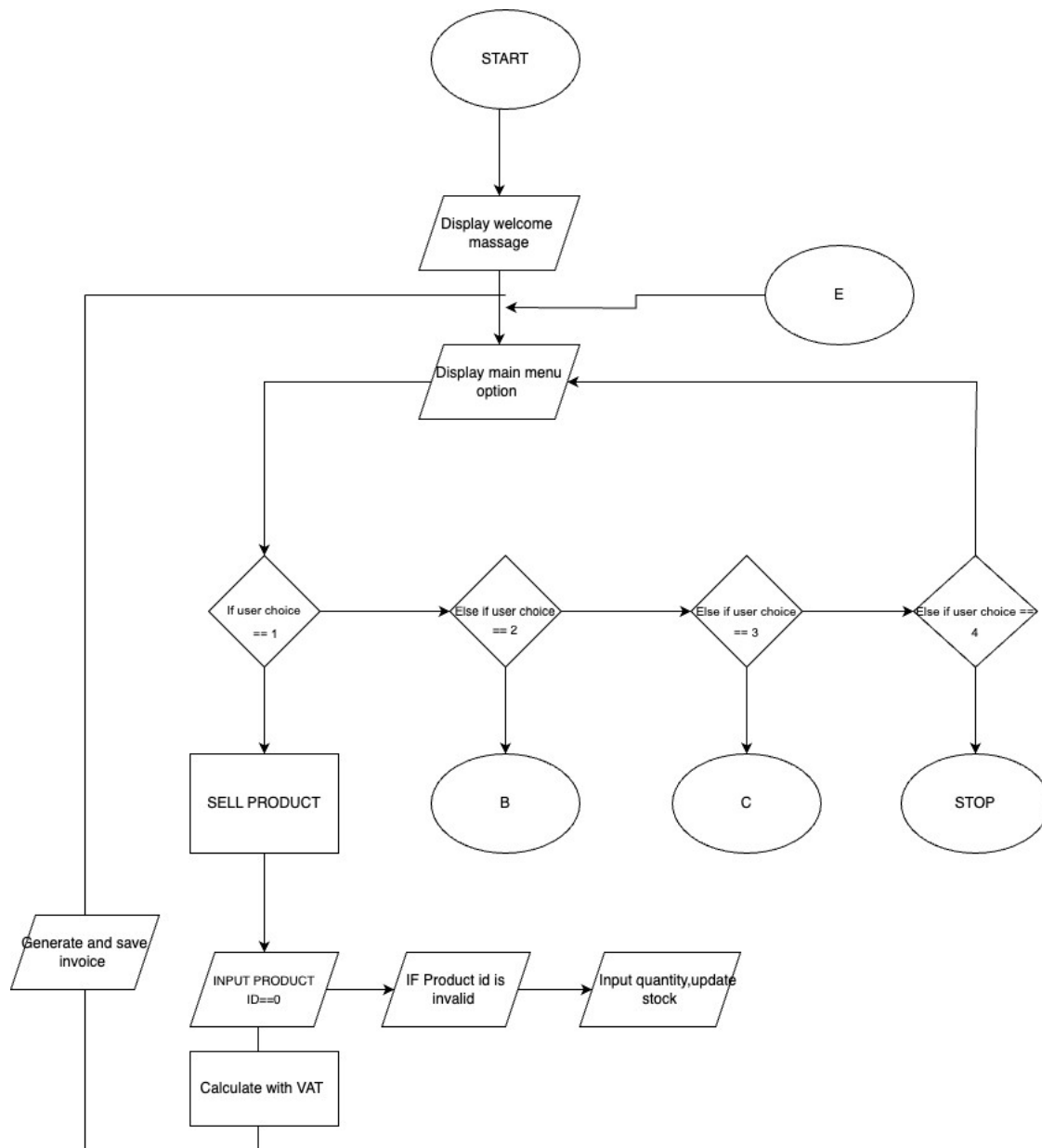


Figure 4 FLOWCHART START

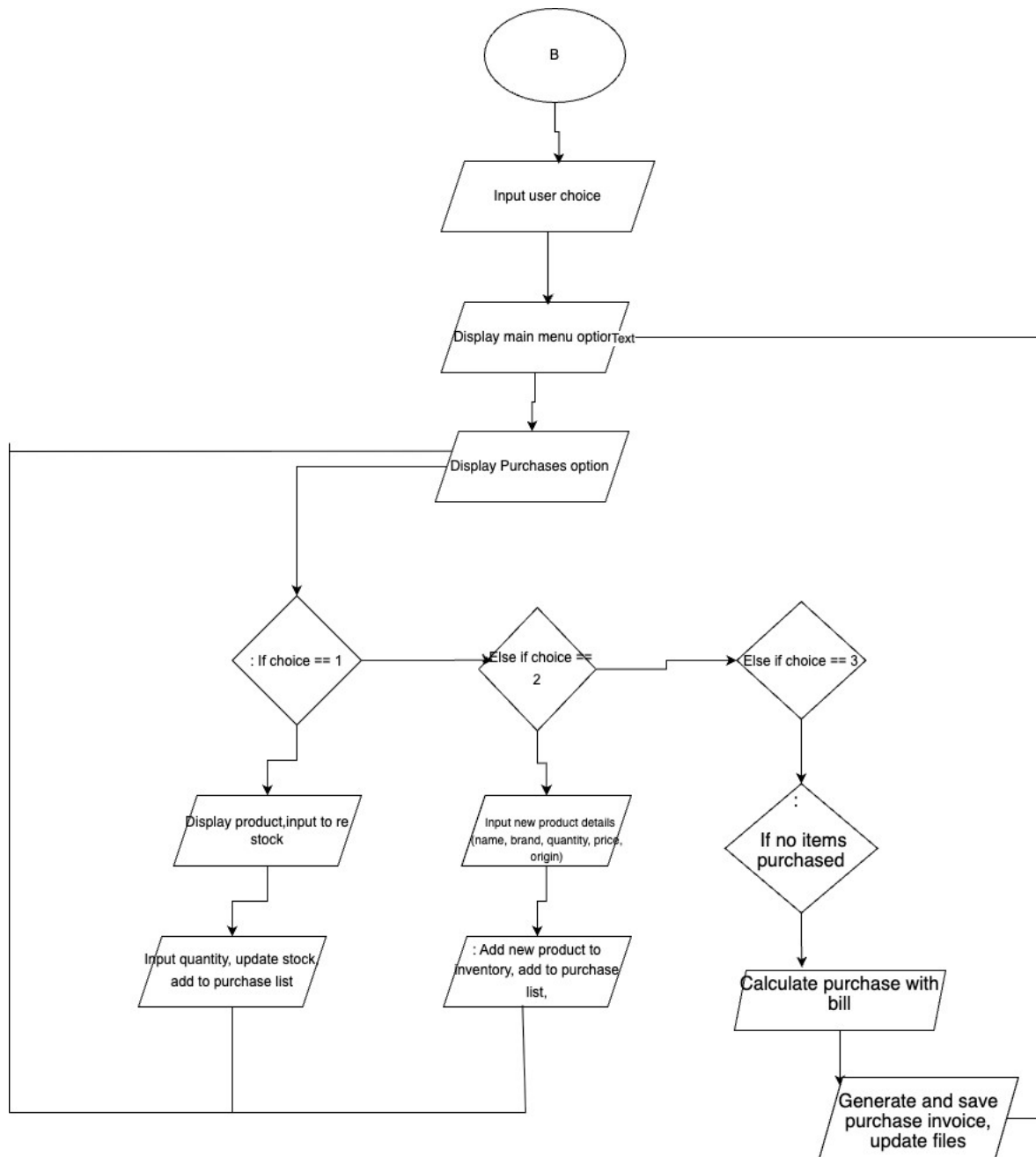


Figure 5 Flowchart for B

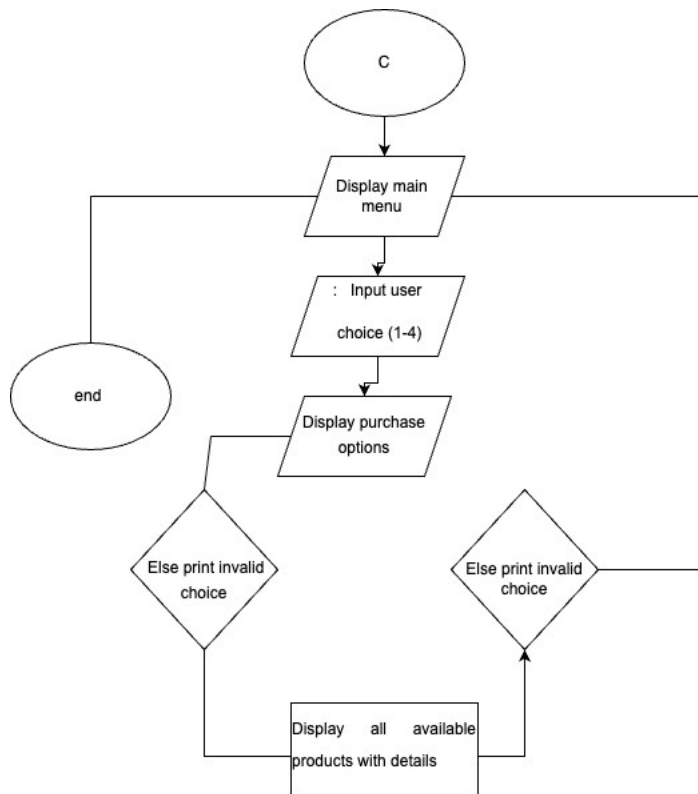


Figure 6 Flowchart for C and END

2.2 Pseudocode for read.py:

DEFINE FUNCTION load_products with one parameter named filename

DEFAULT "Products.txt"

START FUNCTION

DECLARE products_data AS empty Dictionary

BEGIN TRY

OPEN filename in read mode and store it in a variable named file

DECLARE product_id_counter TO 1

FOR line in file, do the following:

IF line **IS NOT** empty AND last character of line **IS** newline **THEN**

line = line without last character

END IF

IF line **IS** empty **THEN**

CONTINUE TO next line

END IF

DECLARE parts **AS** result of **SPLITTING** line by ","

IF LENGTH of parts **IS** 5 **THEN**

```
DECLARE name = parts[0]

DECLARE brand = parts[1]

BEGIN TRY

    DECLARE quantity = CONVERT parts[2] TO Integer

    DECLARE price = CONVERT parts[3] TO Float

    CATCH ValueError

        PRINT "Warning: Invalid quantity or price for product data: " + line +
        ". Skipping."

        CONTINUE TO next line

    END TRY

    DECLARE origin = parts[4]                products_data[product_id_counter]

    = [name, brand, quantity, price, origin]

    INCREMENT product_id_counter

ELSE

    PRINT "Warning: Malformed line in " + filename + ": " + line + ".
    Skipping."

END IF

END FOR

CATCH FileNotFoundError

    PRINT "Error: " + filename + " not found. Starting with an empty inventory."
CATCH Exception AS e
```

PRINT "An unexpected error occurred while loading products: " + **CONVERT**
 e TO String

END TRY

RETURN products_data

END FUNCTION

DEFINE FUNCTION read_bill_counter with one parameter named filename
DEFAULT "bill_counter.txt"

START FUNCTION

BEGIN TRY

 OPEN filename in read mode and store it in a variable named file

DECLARE counter_line = **READ_LINE** from file

IF counter_line **IS NOT** empty **AND** last character of counter_line **IS** newline
 THEN

 counter_line = counter_line without last character

END IF

IF counter_line **IS** empty **THEN**

PRINT "Warning: " + filename + " is empty. Resetting counter to 0."
 RETURN 0

END IF

```
DECLARE counter = CONVERT counter_line TO Integer

RETURN counter

CATCH FileNotFoundError

    RETURN 0

CATCH ValueError

    PRINT "Warning: " + filename + " contains invalid data. Resetting counter to
0."

    RETURN 0

CATCH Exception AS e

    PRINT "Error reading bill counter from " + filename + ": " + CONVERT e TO
String

    RETURN 0

END TRY

END FUNCTION
```

2.3 Pseudocode for write.py:

IMPORT datetime_module // To access current time functions

DEFINE FUNCTION save_products with two parameters named filename, products_data

START FUNCTION

BEGIN TRY

OPEN filename in write mode and store it in a variable named file

DECLARE product_ids_to_write = **CONVERT** keys of products_data **TO** List

DECLARE n = **LENGTH** of **product_ids_to_write**

FOR i **FROM** 0 **TO** n-1, do the following:

FOR j **FROM** 0 **TO** n-i-2, do the following: // n-i-1 in Python for 0-indexed end

IF **CONVERT** product_ids_to_write[j] **TO** Integer **>**

CONVERT product_ids_to_write[j+1] **TO** Integer **THEN**

SWAP product_ids_to_write[j] **AND** product_ids_to_write[j+1]

END IF

END FOR

END FOR

FOR product_id in product_ids_to_write, do the following:

DECLARE details = products_data[product_id]

SUPREM THAPA

```
DECLARE line_parts = [  
  
    CONVERT details[0] TO String, // name  
  
    CONVERT details[1] TO String, // brand  
  
    CONVERT details[2] TO String, // quantity  
  
    FORMAT details[3] TO String with 2 decimal places, // price  
  
    CONVERT details[4] TO String // origin  
  
    ]  
  
    WRITE (JOIN line_parts with ",") + newline character TO file  
  
    END FOR  
  
    CATCH Exception AS e  
  
        PRINT "Error saving products to " + filename + ": " + CONVERT e TO String  
  
    END TRY  
  
END FUNCTION  
  
  
DEFINE FUNCTION write_bill_counter with two parameters named new_counter,  
filename DEFAULT "bill_counter.txt"  
  
START FUNCTION  
  
    BEGIN TRY  
  
        OPEN filename in write mode and store it in a variable named file  
  
        WRITE CONVERT new_counter TO String TO file  
  
    SUPREM THAPA
```

CATCH Exception AS e

PRINT "Error writing to " + filename + ": " + **CONVERT** e **TO** String

END TRY

END FUNCTION

DEFINE FUNCTION generate_unique_bill_id with one parameter named counter

START FUNCTION

DECLARE now = **CALL** datetime_module.getCurrentDateTime() // Assumes a function like datetime.datetime.now()

DECLARE year_str = **CONVERT** now.year **TO** String

DECLARE month_val = now.month

DECLARE month_str = **CONVERT** month_val **TO** String

IF month_val < 10 **THEN** month_str = "0" + month_str

END IF

DECLARE day_val = now.day

DECLARE day_str = **CONVERT** day_val **TO** String

IF day_val < 10 **THEN** day_str = "0" + day_str

END IF

SUPREM THAPA

RETURN year_str + month_str + day_str + "_" + CONVERT counter TO String

END FUNCTION

DEFINE FUNCTION print_and_save_bill with parameters bill_id, customer_name,
items_list, subtotal, discount_amount, vat_amount, grand_total,
bill_type, company_name, company_address

START FUNCTION

DECLARE bill_content_list AS empty List

APPEND two newline characters TO bill_content_list

APPEND tab characters + company_name + newline character TO bill_content_list

APPEND tab characters + company_address + newline character TO bill_content_list

APPEND newline character TO bill_content_list

IF bill_type IS "SALE" **THEN**

APPEND "----- SALE INVOICE -----" + newline character TO
bill_content_list

ELSE IF bill_type IS "PURCHASE" **THEN**

APPEND "----- PURCHASE INVOICE -----" + newline character
TO bill_content_list

SUPREM THAPA

END IF

DECLARE now = CALL datetime_module.getCurrentDateTime()

// Manual date formatting (year-month-day)

// ... (logic for year_str, month_str, day_str as in generate_unique_bill_id) ...

DECLARE date_str = year_str + "-" + month_str + "-" + day_str

// Manual time formatting (hour:minute:second)

// ... (logic for hour_str, minute_str, second_str similar to date) ...

DECLARE time_str = hour_str + ":" + minute_str + ":" + second_str

APPEND "Bill ID: " + bill_id + newline character TO bill_content_list

APPEND "Date: " + date_str + tab character + "Time: " + time_str + two newline characters TO bill_content_list

IF bill_type IS "SALE" THEN

APPEND "Customer Name: " + customer_name + two newline characters TO bill_content_list

ELSE IF bill_type IS "PURCHASE" THEN

 APPEND "Supplier: Manufacturer/Internal" + two newline characters TO bill_content_list

END IF

SUPREM THAPA

DECLARE header_line = "S.N." + tab + "Product Name" + tab + "Qty" + tab +
"Unit Price" + tab + "Total Price" + newline

DECLARE separator_line = REPEAT "-" 65 times + newline character

APPEND separator_line TO bill_content_list

APPEND header_line TO bill_content_list

APPEND separator_line TO bill_content_list

DECLARE item_serial_number = 1

FOR item in items_list, do the following:

DECLARE name_col = item['name']

DECLARE max_name_len = 20

IF LENGTH of name_col > max_name_len THEN name_col =

SUBSTRING of name_col from start to max_name_len-4 + "..." ELSE

name_col = name_col + REPEAT " " (max_name_len - LENGTH of name_col)
times

END IF

DECLARE unit_price_str = **FORMAT** item['price_per_unit'] TO String with 2 decimal
places

SUPREM THAPA

DECLARE item_total_str = **FORMAT** item['total'] TO String with 2 decimal places

DECLARE line = CONVERT item_serial_number TO String + tab character +

name_col + tab character +

CONVERT item['quantity'] TO String + tab character +

unit_price_str + two tab characters + // Adjust tabs for alignment

item_total_str + newline character

APPEND line TO bill_content_list

INCREMENT item_serial_number

END FOR

APPEND separator_line TO bill_content_list

IF bill_type IS "SALE" THEN

APPEND tabs + "Subtotal: " + tab + "Rs. " + **FORMAT** subtotal TO String with 2 decimal places + newline TO bill_content_list

IF discount_amount > 0 THEN

APPEND tabs + "Clearance Discount (10%): Rs. " + **FORMAT** discount_amount TO String with 2 decimal places + newline TO bill_content_list

END IF

SUPREM THAPA

DECLARE net_amount_for_display = subtotal - discount_amount

APPEND tabs + "Net Amount: " + tab + "Rs. " + **FORMAT** net_amount_for_display
TO String with 2 decimal places + newline TO bill_content_list

APPEND tabs + "VAT (13%): " + tab + "Rs. " + **FORMAT** vat_amount TO String with
2 decimal places + newline TO bill_content_list

ELSE IF bill_type IS "PURCHASE" THEN

APPEND tabs + "Subtotal: " + tab + "Rs. " + **FORMAT** subtotal TO String with 2
decimal places + newline TO bill_content_list

END IF

APPEND separator_line TO bill_content_list

APPEND tabs + "GRAND TOTAL: " + tab + "Rs. " + **FORMAT** grand_total TO String
with 2 decimal places + newline TO bill_content_list

APPEND separator_line TO bill_content_list

IF bill_type IS "SALE" THEN

APPEND newline + "Thank you for your purchase, " + customer_name + "!" +
newline TO bill_content_list

ELSE IF bill_type IS "PURCHASE" THEN

APPEND newline + "Items purchased/restocked successfully." + newline TO
bill_content_list

END IF

SUPREM THAPA

APPEND two newline characters TO bill_content_list

DECLARE final_bill_string = JOIN all elements in bill_content_list

PRINT final_bill_string

IF bill_type IS "SALE" THEN

DECLARE bill_filename_prefix = "SALE_BILL_"

ELSE

DECLARE bill_filename_prefix = "PURCHASE_BILL_"

END IF

DECLARE bill_filename = bill_filename_prefix + bill_id + ".txt"

BEGIN TRY

 OPEN bill_filename in write mode and store it in a variable named file

WRITE final_bill_string TO file

 PRINT "Bill successfully saved as " + bill_filename

CATCH Exception AS e

 PRINT "Error saving bill to file " + bill_filename + ": " + CONVERT e TO String

END TRY

END FUNCTION

SUPREM THAPA

2.4 Pseudocode for operations.py

DEFINE FUNCTION display_products with one parameter named products_data

START FUNCTION

PRINT newline + "----- **AVAILABLE PRODUCTS** -----"

DECLARE header = "ID" + tab + "Name" " + tab + "Brand" " + tab +
"Qty" + tab + "Price (Cost)" + tab + "Origin"

PRINT header

PRINT REPEAT "-" (LENGTH of header + 15) times

IF products_data IS empty THEN

PRINT "No products available in the inventory."

PRINT "-----"

RETURN

END IF

DECLARE ids_to_display = CONVERT keys of products_data TO List

DECLARE n = LENGTH of ids_to_display

FOR i FROM 0 TO n-1, do the following:

DECLARE min_idx = i

FOR j FROM i+1 TO n-1, do the following:

IF CONVERT ids_to_display[j] TO Integer < CONVERT ids_to_display[min_idx]

TO Integer THEN

min_idx = j

END IF

END FOR

SWAP ids_to_display[i] AND ids_to_display[min_idx]

END FOR

SUPREM THAPA

FOR product_id in ids_to_display, do the following:

 DECLARE details = products_data[product_id]

 DECLARE name = details[0]

 DECLARE brand = details[1]

 DECLARE quantity = details[2]

 DECLARE cost_price = details[3]

 DECLARE origin = details[4]

 DECLARE name_col_width = 22

 DECLARE brand_col_width = 22

 // ... (logic for name_display and brand_display padding/truncation) ...

 DECLARE price_str = FORMAT cost_price TO String with 2 decimal places

 PRINT CONVERT product_id TO String + tab + name_display + tab + brand_display
+ tab +
 CONVERT quantity TO String + tab + price_str + two tabs + origin
END FOR
PRINT "-----"
END FUNCTION

DEFINE FUNCTION get_valid_product_id with three parameters named products_data,
prompt_message, allow_zero DEFAULT False

START FUNCTION

 WHILE True, do the following:

 BEGIN TRY

 INPUT prompt_message INTO pid_str

 DECLARE pid = CONVERT pid_str TO Integer

 IF allow_zero IS True AND pid IS 0 THEN

 RETURN 0

 END IF

 IF pid IS NOT a key in products_data THEN // Assumes integer keys

SUPREM THAPA

```

        PRINT "Error: Product ID not found. Please enter a valid ID from the list."
    ELSE
        RETURN pid
    END IF
CATCH ValueError
    PRINT "Error: Invalid input. Please enter a numerical Product ID."
CATCH Exception AS e
    PRINT "An unexpected error occurred with product ID input: " + CONVERT e TO
String
    END TRY
END WHILE
END FUNCTION

```

DEFINE FUNCTION get_valid_quantity with one parameter named prompt_message
 START FUNCTION

```

    WHILE True, do the following:
        BEGIN TRY
            INPUT prompt_message INTO qty_str
            DECLARE qty = CONVERT qty_str TO Integer
            IF qty <= 0 THEN
                PRINT "Error: Quantity must be a positive whole number."
            ELSE
                RETURN qty
            END IF
        CATCH ValueError
            PRINT "Error: Invalid input. Please enter a numerical quantity."
        CATCH Exception AS e
            PRINT "An unexpected error occurred with quantity input: " + CONVERT e TO
String
        END TRY
    END WHILE
END FUNCTION

```

DEFINE FUNCTION sell_products with one parameter named products_data
 START FUNCTION

 IF products_data IS empty THEN

 PRINT "Inventory is empty. Cannot sell products."

 RETURN products_data, Null // Null represents no bill details

 END IF

 INPUT "Enter Customer Name: " INTO customer_name

 DECLARE sold_items_for_bill AS empty List

 DECLARE current_sale_subtotal = 0.0

 DECLARE apply_clearance_discount = False

 WHILE True, do the following:

 CALL display_products(products_data)

 DECLARE product_id = CALL get_valid_product_id(products_data, "Enter Product
 ID to sell (or 0 to finish current sale): ", True)

 IF product_id IS 0 THEN

 IF sold_items_for_bill IS empty THEN

 PRINT "No products were selected for this sale."

 RETURN products_data, Null

 END IF

 BREAK WHILE // Exit loop to proceed to bill calculation

 END IF

 DECLARE product_details = products_data[product_id]

 DECLARE product_name = product_details[0]

 DECLARE current_stock = product_details[2]

 DECLARE cost_price = product_details[3]

 DECLARE selling_price_per_unit = cost_price * 2.0

SUPREM THAPA

```
PRINT "Selected: " + product_name + " (Stock: " + CONVERT current_stock TO
String + ", Selling Price/unit: Rs. " + FORMAT selling_price_per_unit TO String with 2
decimal places + ")"
```

```
DECLARE requested_quantity = CALL get_valid_quantity("Enter quantity to sell: ")
```

```
DECLARE free_items = requested_quantity INTEGER_DIVIDE 3
```

```
DECLARE total_quantity_to_deduct_from_stock = requested_quantity + free_items
```

```
IF total_quantity_to_deduct_from_stock > current_stock THEN
```

```
    PRINT "Error: Not enough stock for " + product_name + "."
```

```
// ... (print available and required stock) ...
```

```
    CONTINUE TO next iteration of WHILE // Ask for another product
```

```
END IF
```

```
products_data[product_id][2]      =      products_data[product_id][2]      -
total_quantity_to_deduct_from_stock
```

```
IF products_data[product_id][2] IS 0 THEN
```

```
    apply_clearance_discount = True
```

```
    PRINT "INFO: Product " + product_name + " is now out of stock. Clearance
discount may apply to the bill."
```

```
END IF
```

```
DECLARE item_sale_total = requested_quantity * selling_price_per_unit
```

```
current_sale_subtotal = current_sale_subtotal + item_sale_total
```

```
APPEND { 'id': product_id, 'name': product_name, 'quantity': requested_quantity,
        'price_per_unit': selling_price_per_unit, 'total': item_sale_total,
        'free_items_received':      free_items,      'total_deducted_stock':
total_quantity_to_deduct_from_stock }
TO sold_items_for_bill
```

```
PRINT CONVERT requested_quantity TO String + " " + product_name + "(s) added
to bill."
```

```
IF free_items > 0 THEN
```

```
PRINT "BONUS: Customer receives " + CONVERT free_items TO String + " " +
product_name + "(s) for free!"
```

```
END IF
```

```
INPUT "Sell another product in this transaction? (yes/no): " INTO
another_item_choice
```

```
IF another_item_choice IS NOT "yes" AND another_item_choice IS NOT "YES" AND
another_item_choice IS NOT "y" AND another_item_choice IS NOT "Y" THEN
BREAK WHILE // Finish adding items for this sale
```

```
END IF
```

```
END WHILE
```

```
IF sold_items_for_bill IS empty THEN
```

```
RETURN products_data, Null
```

```
END IF
```

```
// Sales Invoice Logic
```

```
DECLARE discount_amount = 0.0 IF
```

```
apply_clearance_discount IS True THEN
```

```
discount_amount = current_sale_subtotal * 0.10
```

```
PRINT "INFO: Applying 10% clearance discount to the subtotal."
```

```
END IF
```

```
DECLARE amount_after_discount = current_sale_subtotal - discount_amount
```

```
DECLARE vat_amount = amount_after_discount * 0.13
```

```
DECLARE grand_total_payable = amount_after_discount + vat_amount
```

```
DECLARE bill_details_for_writing = {
```

```
"customer_name": customer_name,
```

```
SUPREM THAPA
```

```

    "items_list": sold_items_for_bill,
    "subtotal": current_sale_subtotal,
    "discount_amount": discount_amount,
    "vat_amount": vat_amount,
    "grand_total": grand_total_payable
}

```

PRINT newline + "--- Sale Transaction Summary ---"

FOR item in sold_items_for_bill, do the following:

```

    PRINT item['name'] + " - Qty Paid: " + CONVERT item['quantity'] TO String + ", Free:
" + CONVERT item['free_items_received'] TO String + ", Total Deducted: " + CONVERT
item['total_deducted_stock'] TO String

```

END FOR

RETURN products_data, bill_details_for_writing

END FUNCTION

DEFINE FUNCTION purchase_products with one parameter named products_data

START FUNCTION

PRINT newline + "--- Purchase from Manufacturer/Supplier ---"

DECLARE purchased_items_for_bill AS empty List

DECLARE current_purchase_total_cost = 0.0

WHILE True, do the following:

PRINT newline + "Purchase Options:"

PRINT "1. Restock an existing product"

PRINT "2. Add a completely new product"

PRINT "3. Finish current purchase transaction"

INPUT "Enter your choice (1-3): " INTO action_choice_str

BEGIN TRY

DECLARE action_choice = CONVERT action_choice_str TO Integer

CATCH ValueError

PRINT "Invalid choice. Please enter a number (1, 2, or 3)."

CONTINUE TO next iteration of WHILE

END TRY

IF action_choice IS 1 THEN // Restock existing

IF products_data IS empty THEN

PRINT "No products exist to restock. Please add a new product first (Option 2)."

CONTINUE TO next iteration of WHILE

END IF

CALL display_products(products_data)

DECLARE product_id = CALL get_valid_product_id(products_data, "Enter Product ID of existing product to restock: ")

// ... (restock logic: get details, quantity, update products_data, calculate item_total_purchase_cost, append to purchased_items_for_bill) ...

PRINT "Items restocked."

ELSE IF action_choice IS 2 THEN // Add new product

PRINT newline + "--- Add New Product to Inventory ---"

// ... (get new product details: name, brand, quantity, cost_price, origin) ...

// ... (generate new_id for product) ...

// ... (add to products_data, calculate item_total_purchase_cost, append to purchased_items_for_bill) ... PRINT "New product added."

ELSE IF action_choice IS 3 THEN // Finish current purchase transaction

IF purchased_items_for_bill IS empty THEN

PRINT "No products were restocked or added in this transaction."

RETURN products_data, Null

END IF

BREAK WHILE // Proceed to bill calculation

ELSE

PRINT "Invalid choice. Please enter 1, 2, or 3."

END IF

SUPREM THAPA

END WHILE

IF purchased_items_for_bill IS empty THEN

 RETURN products_data, Null

END IF

// Purchase Invoice Logic

DECLARE discount_amount = 0.0

DECLARE vat_amount = 0.0

DECLARE grand_total_paid_by_store = current_purchase_total_cost

DECLARE bill_details_for_writing = {

 "customer_name": "Store Purchase (Self/Supplier)",

 "items_list": purchased_items_for_bill,

 "subtotal": current_purchase_total_cost,

 "discount_amount": discount_amount,

 "vat_amount": vat_amount,

 "grand_total": grand_total_paid_by_store

}

RETURN products_data, bill_details_for_writing

END FUNCTION

2.5 Pseudocode for main.py

IMPORT read_module

IMPORT write_module

IMPORT operations_module

SUPREM THAPA

```

DECLARE COMPANY_NAME = "TopskinCare Wholesale"
DECLARE COMPANY_ADDRESS = "Madhyapur, Thimi, Bhaktapur"
DECLARE PRODUCTS_FILENAME = "Products.txt"
DECLARE BILL_COUNTER_FILENAME = "bill_counter.txt"

```

```

DEFINE FUNCTION display_welcome_message
START FUNCTION
    PRINT two newline characters
    PRINT tab characters + COMPANY_NAME
    PRINT newline character
    PRINT tab characters + COMPANY_ADDRESS
    PRINT newline character
END FUNCTION

```

```

DEFINE FUNCTION main_program_loop
START FUNCTION
    CALL display_welcome_message()

```

```

DECLARE                    products_inventory                    =                    CALL
read_module.load_products(PRODUCTS_FILENAME)
DECLARE system_active = True

```

```

WHILE system_active IS True, do the following:
    PRINT newline + "===== Main Menu ====="
    PRINT "1. Sell Product(s) to Customer"
    PRINT "2. Purchase Product(s) (Restock/Add New)"
    PRINT "3. Display All Available Products"
    PRINT "4. Exit System"
    PRINT "===== "

```

```

    INPUT "Enter your choice (1-4): " INTO user_choice_str
SUPREM THAPA

```

```
BEGIN TRY
```

```
    DECLARE main_menu_choice = CONVERT user_choice_str TO Integer
```

```
CATCH ValueError
```

```
    PRINT "Error: Invalid input! Please enter a number between 1 and 4."
```

```
    CONTINUE TO next iteration of WHILE
```

```
CATCH Exception AS e
```

```
    PRINT "An unexpected error occurred with your input: " + CONVERT e TO String
```

```
    CONTINUE TO next iteration of WHILE
```

```
END TRY
```

```
IF main_menu_choice IS 1 THEN // Sell Products
```

```
    DECLARE updated_inventory_after_sale, sale_bill_details = CALL
operations_module.sell_products(products_inventory) products_inventory =
updated_inventory_after_sale
```

```
    IF sale_bill_details IS NOT Null THEN
```

```
        DECLARE current_bill_num = CALL
read_module.read_bill_counter(BILL_COUNTER_FILENAME)
```

```
        DECLARE new_bill_num = current_bill_num + 1
```

```
        DECLARE unique_bill_id = CALL
write_module.generate_unique_bill_id(new_bill_num)
```

```
        CALL write_module.print_and_save_bill(
unique_bill_id, sale_bill_details["customer_name"],
sale_bill_details["items_list"],
sale_bill_details["subtotal"],
sale_bill_details["discount_amount"],
sale_bill_details["vat_amount"],
sale_bill_details["grand_total"],
```

```
        "SALE",
```

```
        COMPANY_NAME,
```

```
SUPREM THAPA
```

```

        COMPANY_ADDRESS
    )
    CALL write_module.save_products(PRODUCTS_FILENAME,
products_inventory)
    CALL                                write_module.write_bill_counter(new_bill_num,
BILL_COUNTER_FILENAME)
    ELSE
        PRINT "Sale operation did not complete or no items were sold. No bill
generated."
    END IF
    ELSE IF main_menu_choice IS 2 THEN // Purchase Products
        DECLARE updated_inventory_after_purchase, purchase_bill_details = CALL
operations_module.purchase_products(products_inventory)      products_inventory =
updated_inventory_after_purchase

        IF purchase_bill_details IS NOT Null THEN
            DECLARE current_bill_num = CALL
read_module.read_bill_counter(BILL_COUNTER_FILENAME)
            DECLARE new_bill_num = current_bill_num + 1
            DECLARE unique_bill_id = CALL
write_module.generate_unique_bill_id(new_bill_num)

            CALL write_module.print_and_save_bill(
unique_bill_id,
purchase_bill_details["customer_name"],
purchase_bill_details["items_list"],
purchase_bill_details["subtotal"],
purchase_bill_details["discount_amount"],
purchase_bill_details["vat_amount"],
purchase_bill_details["grand_total"],
            "PURCHASE",
            COMPANY_NAME,

```

```

        COMPANY_ADDRESS
    )
    CALL write_module.write_bill_counter(new_bill_num,
BILL_COUNTER_FILENAME)
    ELSE
        PRINT "Purchase operation did not complete with billable items or was
cancelled."
    END IF
    CALL write_module.save_products(PRODUCTS_FILENAME, products_inventory)

    ELSE IF main_menu_choice IS 3 THEN // Display Products
        CALL operations_module.display_products(products_inventory)

    ELSE IF main_menu_choice IS 4 THEN // Exit
        PRINT newline + "Thank you for using the " + COMPANY_NAME + " system."
    PRINT "Exiting now..."      system_active = False
    ELSE
        PRINT "Error: Invalid choice. Please select an option from 1 to 4."
    END IF
    END WHILE
END FUNCTION

// Simulating Python's __name__ == "__main__":
IF this script is the main one being run THEN
    CALL main_program_loop()
END IF

```

2.6 DATA STRUCTURE

. Primary Data Structure: Dictionary for Product Inventory

- Implemented Data Structure: The core of the inventory management is a Python dictionary, where keys are integer product IDs and values are lists containing product details.
- Screenshot of the code (from `read.py` - loading part):

```
# read.py
def load_products(filename="Products.txt"):
    products_data = {} # <<<< DICTIONARY INITIALIZATION
    try:
        with open(filename, "r") as file:
            product_id_counter = 1
            for line in file:
                # ... (line processing) ...
                if len(parts) == 5:
                    # ... (type conversions) ...
                    # <<<< STORING DATA IN DICTIONARY
                    products_data[product_id_counter] = [name, brand, quantity, price, origin]
                    product_id_counter += 1
                # ...
            # ... (exception handling) ...
    return products_data
```

Figure 7 READ.PY EXAMPLE

Explanation:

- In `products_data = {}`, an empty dictionary is created.

- Inside the loop, `products_data[product_id_counter] = [name, brand, quantity, price, origin]` demonstrates how data is stored.
 - `product_id_counter` (an integer) serves as the unique key.
- `[name, brand, quantity, price, origin]` (a list of product attributes) serves as the value associated with that key.

Advantages:

- Efficient Lookup: Accessing a product by its ID (key) is very fast (average $O(1)$ time complexity).
- Dynamic: Can easily add new products or modify existing ones.
- Associative: Clearly links a unique ID to its set of details..

Lists (general purpose):

- Example: `product_ids_to_write = list(products_data.keys())` in `write.save_products`.
- Description: Used to hold a sequence of items, often for iteration or sorting (like sorting product IDs before writing to the file).
- Why this structure? Lists are Python's fundamental ordered, mutable sequence type, ideal for such tasks.

Strings:

- Description: Used extensively for product names, brands, origins, customer details, file content, user input, and messages displayed to the user.
- Example: `line_parts` in `save_products` (a list of strings to be joined), `customer_name`.

Integers and Floats:

- Description: Used for numerical data such as product IDs, quantities (integers), prices, subtotals, VAT, discounts (floats), and the bill counter (integer).
- Example: `product_id_counter`, `quantity`, `price`, `subtotal`.

Tuple

- A tuple is one of Python's built-in sequence data types. At its core, a tuple is an ordered, immutable collection of heterogeneous elements.

3 PROGRAM

Implementation of the program (Overall program with short explanation)

```

# read.py

def load_products(filename="Products.txt"):
    """
    Loads product data from a specified file.
    Each line in the file should be: name,brand,quantity,price,origin
    Returns a dictionary of products.
    """
    products_data = {}
    try:
        with open(filename, "r") as file:
            product_id_counter = 1
            for line in file:
                # Manually remove newline character if present at the end of the line
                if line and line[-1] == '\n': # Check if line is not empty before accessing -1
                    line = line[:-1]

                if not line: # Skip empty lines
                    continue

                parts = line.split(",")
                if len(parts) == 5:
                    name = parts[0]
                    brand = parts[1]
                    try:
                        quantity = int(parts[2])
                        price = float(parts[3])
                    except ValueError:
                        # Using string concatenation as f-strings/format are disallowed
                        print("Warning: Invalid quantity or price for product data: '" + line + "'. Skipping.")
                        continue
                    origin = parts[4]
                    products_data[product_id_counter] = [name, brand, quantity, price, origin]
                    product_id_counter += 1
                else:
                    print("Warning: Malformed line in " + filename + ": '" + line + "'. Skipping.")
    except FileNotFoundError:
        print("Error: " + filename + " not found. Starting with an empty inventory.")
    except Exception as e:
        # Using str(e) for error message
        print("An unexpected error occurred while loading products: " + str(e))
    return products_data

def read_bill_counter(filename="bill_counter.txt"):
    """
    Reads the current bill counter value from its file.
    Returns the counter value, or 0 if file not found or invalid.
    """
    try:
        with open(filename, "r") as file:
            counter_line = file.readline()
            # Remove newline if present, as int() can fail with it. No strip() allowed.
            if counter_line and counter_line[-1] == '\n':
                counter_line = counter_line[:-1]

            if not counter_line: # Handle empty file or line
                print("Warning: " + filename + " is empty. Resetting counter to 0.")
                return 0

            counter = int(counter_line)
            return counter
    except FileNotFoundError:
        return 0 # Start from 0 if file doesn't exist
    except ValueError:
        print("Warning: " + filename + " contains invalid data. Resetting counter to 0.")
        return 0
    except Exception as e:
        print("Error reading bill counter from " + filename + ": " + str(e))
        return 0

```

Figure 8 Code of READ.PY

```
# write.py
import datetime # Allowed for datetime.datetime.now() parts, not strftime

def save_products(filename, products_data):
    """
    Saves the current product data back to the specified file.
    """
    try:
        with open(filename, "w") as file:
            product_ids_to_write = list(products_data.keys())
            n = len(product_ids_to_write)
            for i in range(n):
                for j in range(0, n-i-1):
                    # Ensure keys are comparable, assuming they are integers
                    if int(product_ids_to_write[j]) > int(product_ids_to_write[j+1]):
                        product_ids_to_write[j], product_ids_to_write[j+1] = product_ids_to_write[j+1], product_ids_to_write[j]

            for product_id in product_ids_to_write:
                details = products_data[product_id]
                line_parts = [
                    str(details[0]), # name
                    str(details[1]), # brand
                    str(details[2]), # quantity
                    "%.2f" % details[3], # price
                    str(details[4]) # origin
                ]
                file.write(",".join(line_parts) + "\n")
    except Exception as e:
        print("Error saving products to " + filename + ": " + str(e))

def write_bill_counter(new_counter, filename="bill_counter.txt"):
    """
    Writes the new bill counter value to its file.
    """
    try:
        with open(filename, "w") as file:
            file.write(str(new_counter))
    except Exception as e:
        print("Error writing to " + filename + ": " + str(e))

def generate_unique_bill_id(counter):
    """
    Generates a unique bill ID using current date/time components and a counter.
    No strftime is used.
    """
    now = datetime.datetime.now()
    year = str(now.year)

    month_val = now.month
    month_str = str(month_val)
    if month_val < 10:
        month_str = "0" + month_str

    day_val = now.day
    day_str = str(day_val)
    if day_val < 10:
        day_str = "0" + day_str

    return year + month_str + day_str + "_" + str(counter)

def print_and_save_bill(bill_id, customer_name, items_list,
                        subtotal, discount_amount, vat_amount, grand_total,
                        bill_type, company_name, company_address):
    """
    Formats, prints to console, and saves a bill to a unique file.
    bill_type can be "SALE" or "PURCHASE".
    items_list is a list of dictionaries, each:
    {'name': str, 'quantity': int, 'price_per_unit': float, 'total': float}
    """
```

Figure 9 Code of WRITE.PY

Figure 10

```

bill_content_list.append(separator_line)

if bill_type == "SALE":
    bill_content_list.append("\t\t\t\tSubtotal: \tRs. " + ("%2f" % subtotal) + "\n")
    if discount_amount > 0: # Only print discount if it's applied
        bill_content_list.append("\t\t\t\tClearance Discount (10%): Rs. " + ("%2f" % discount_amount) + "\n")
    net_amount_for_display = subtotal - discount_amount
    bill_content_list.append("\t\t\t\tNet Amount: \tRs. " + ("%2f" % net_amount_for_display) + "\n")
    bill_content_list.append("\t\t\t\tVAT (13%): \tRs. " + ("%2f" % vat_amount) + "\n")
elif bill_type == "PURCHASE":
    bill_content_list.append("\t\t\t\tSubtotal: \tRs. " + ("%2f" % subtotal) + "\n")
    # No Discount or VAT lines for purchase from manufacturer

bill_content_list.append(separator_line)
bill_content_list.append("\t\t\t\tGRAND TOTAL: \tRs. " + ("%2f" % grand_total) + "\n")
bill_content_list.append(separator_line)

if bill_type == "SALE":
    bill_content_list.append("\nThank you for your purchase, " + customer_name + "!\n")
elif bill_type == "PURCHASE":
    bill_content_list.append("\nItems purchased/restocked successfully.\n")
bill_content_list.append("\n" * 2)

final_bill_string = "".join(bill_content_list)
print(final_bill_string)

bill_filename_prefix = "SALE_BILL_" if bill_type == "SALE" else "PURCHASE_BILL_"
bill_filename = bill_filename_prefix + bill_id + ".txt"
try:
    with open(bill_filename, "w") as file:
        file.write(final_bill_string)
    print("Bill successfully saved as " + bill_filename)
except Exception as e:
    print("Error saving bill to file " + bill_filename + ": " + str(e))

```

Figure 11

```
# operations.py

def display_products(products_data):
    """
    Displays available products in a formatted manner.
    """
    print("\n----- AVAILABLE PRODUCTS -----")
    header = "ID\tName\tBrand\tQty\tPrice (Cost)\tOrigin"
    print(header)
    print("-" * (len(header) + 15))

    if not products_data:
        print("No products available in the inventory.")
        print("-----")
        return

    ids_to_display = list(products_data.keys())
    n = len(ids_to_display)
    for i in range(n):
        min_idx = i
        for j in range(i + 1, n):
            if int(ids_to_display[j]) < int(ids_to_display[min_idx]): # Ensure comparison as int
                min_idx = j
        ids_to_display[i], ids_to_display[min_idx] = ids_to_display[min_idx], ids_to_display[i]

    for product_id in ids_to_display:
        details = products_data[product_id]
        name = details[0]
        brand = details[1]
        quantity = details[2]
        cost_price = details[3]
        origin = details[4]

        name_col_width = 22
        brand_col_width = 22

        name_display = name
        if len(name) > name_col_width:
            name_display = name[:name_col_width-3] + "..."
        else:
            name_display = name + " " * (name_col_width - len(name))

        brand_display = brand
        if len(brand) > brand_col_width:
            brand_display = brand[:brand_col_width-3] + "..."
        else:
            brand_display = brand + " " * (brand_col_width - len(brand))

        price_str = "%.2f" % cost_price

        print(str(product_id) + "\t" + name_display + "\t" + brand_display + "\t" + \
              str(quantity) + "\t" + price_str + "\t\t" + origin)
    print("-----")

def get_valid_product_id(products_data, prompt_message, allow_zero=False):
    """
    Gets a valid product ID from user input.
    If allow_zero is True, 0 is considered a valid input (e.g., to stop adding items).
    """
    while True:
        try:
            pid_str = input(prompt_message)
            pid = int(pid_str)
            if allow_zero and pid == 0:
                return 0
            if pid not in products_data: # Assumes keys are integers
                print("Error: Product ID not found. Please enter a valid ID from the list.")
        except ValueError:
            pass
```

Figure 12 Code of OPERATIONS.PY

```

        else:
            return pid
    except ValueError:
        print("Error: Invalid input. Please enter a numerical Product ID.")
    except Exception as e:
        print("An unexpected error occurred with product ID input: " + str(e))

def get_valid_quantity(prompt_message):
    """
    Gets a valid positive integer quantity from user input.
    """
    while True:
        try:
            qty_str = input(prompt_message)
            qty = int(qty_str)
            if qty <= 0:
                print("Error: Quantity must be a positive whole number.")
            else:
                return qty
        except ValueError:
            print("Error: Invalid input. Please enter a numerical quantity.")
        except Exception as e:
            print("An unexpected error occurred with quantity input: " + str(e))

def sell_products(products_data):
    """
    Handles the process of selling one or more products to a customer.
    Updates product_data in memory and returns it along with bill details.
    """
    if not products_data:
        print("Inventory is empty. Cannot sell products.")
        return products_data, None

    customer_name = input("Enter Customer Name: ")

    sold_items_for_bill = []
    current_sale_subtotal = 0.0
    apply_clearance_discount = False # Flag to check if any item goes out of stock

    while True:
        display_products(products_data)
        product_id = get_valid_product_id(products_data, "Enter Product ID to sell (or 0 to finish current sale): ", allow_zero=True)

        if product_id == 0:
            if not sold_items_for_bill:
                print("No products were selected for this sale.")
                return products_data, None
            break

        product_details = products_data[product_id]
        product_name = product_details[0]
        current_stock = product_details[2]
        cost_price = product_details[3]
        selling_price_per_unit = cost_price * 2.0

        print("Selected: " + product_name + " (Stock: " + str(current_stock) + ", Selling Price/unit: Rs. " + "%.2f" % selling_price_per_unit + ")")
        requested_quantity = get_valid_quantity("Enter quantity to sell: ")

        free_items = requested_quantity // 3
        total_quantity_to_deduct_from_stock = requested_quantity + free_items

        if total_quantity_to_deduct_from_stock > current_stock:
            print("Error: Not enough stock for " + product_name + ".")

```

Figure 13

```

if total_quantity_to_deduct_from_stock > current_stock:
    print("Error: Not enough stock for " + product_name + ".")
    print("Available: " + str(current_stock) + ", Required (incl. free): " + str(total_quantity_to_deduct_from_stock) + ". Please try a smaller quantity.")
    continue

products_data[product_id][2] -= total_quantity_to_deduct_from_stock
if products_data[product_id][2] == 0: # Check if stock became 0
    apply_clearance_discount = True
    print("INFO: Product " + product_name + " is now out of stock. Clearance discount may apply to the bill.")

item_sale_total = requested_quantity * selling_price_per_unit
current_sale_subtotal += item_sale_total

sold_items_for_bill.append({
    'id': product_id,
    'name': product_name,
    'quantity': requested_quantity,
    'price_per_unit': selling_price_per_unit,
    'total': item_sale_total,
    'free_items_received': free_items,
    'total_deducted_stock': total_quantity_to_deduct_from_stock
})

print(str(requested_quantity) + " " + product_name + "(s) added to bill.")
if free_items > 0:
    print("BONUS: Customer receives " + str(free_items) + " " + product_name + "(s) for free!")

another_item_choice = input("Sell another product in this transaction? (yes/no): ")
if not (another_item_choice == "yes" or another_item_choice == "YES" or \
        another_item_choice == "y" or another_item_choice == "Y"):
    break

if not sold_items_for_bill:
    return products_data, None

# Sales Invoice Logic
discount_amount = 0.0
if apply_clearance_discount:
    discount_amount = current_sale_subtotal * 0.10
    print("INFO: Applying 10% clearance discount to the subtotal.")

amount_after_discount = current_sale_subtotal - discount_amount
vat_amount = amount_after_discount * 0.13 # VAT is 13% of the (Subtotal - Discount)
grand_total_payable = amount_after_discount + vat_amount

bill_details_for_writing = {
    "customer_name": customer_name,
    "items_list": sold_items_for_bill,
    "subtotal": current_sale_subtotal,
    "discount_amount": discount_amount,
    "vat_amount": vat_amount,
    "grand_total": grand_total_payable,
}

print("\n--- Sale Transaction Summary ---")
for item in sold_items_for_bill:
    print(item['name'] + " - Qty Paid: " + str(item['quantity']) + ", Free: " + str(item['free_items_received']) + ", Total Deducted: " + str(item['total_deducted_stock']))

return products_data, bill_details_for_writing

def purchase_products(products_data):
    """
    Handles purchasing products: restocking existing or adding new ones.
    Updates product_data in memory and returns it along with bill details.
    """

```

Figure 14

```

print("\n--- Purchase from Manufacturer/Supplier ---")

purchased_items_for_bill = []
current_purchase_total_cost = 0.0

while True:
    print("\nPurchase Options:")
    print("1. Restock an existing product")
    print("2. Add a completely new product")
    print("3. Finish current purchase transaction")

    action_choice_str = input("Enter your choice (1-3): ")
    try:
        action_choice = int(action_choice_str)
    except ValueError:
        print("Invalid choice. Please enter a number (1, 2, or 3).")
        continue

    if action_choice == 1:
        if not products_data:
            print("No products exist to restock. Please add a new product first (Option 2).")
            continue

        display_products(products_data)
        product_id = get_valid_product_id(products_data, "Enter Product ID of existing product to restock: ")

        product_details = products_data[product_id]
        product_name = product_details[0]
        cost_price = product_details[3]

        print("Selected for restock: " + product_name + " (Current Stock: " + str(product_details[2]) + ", Cost Price/unit: Rs. " + ("%2f" % cost_price) + ")")
        quantity_to_add = get_valid_quantity("Enter quantity to add to stock: ")

        products_data[product_id][2] += quantity_to_add
        item_total_purchase_cost = quantity_to_add * cost_price
        current_purchase_total_cost += item_total_purchase_cost

        purchased_items_for_bill.append({
            'id': product_id,
            'name': product_name,
            'brand': product_details[1],
            'quantity': quantity_to_add,
            'price_per_unit': cost_price,
            'total': item_total_purchase_cost
        })
        print(str(quantity_to_add) + " " + product_name + "(s) restocked. New stock: " + str(products_data[product_id][2]))

    elif action_choice == 2:
        print("\n--- Add New Product to Inventory ---")
        new_product_name = input("Enter new product's Name: ")
        new_product_brand = input("Enter new product's Brand: ")
        new_product_quantity = get_valid_quantity("Enter initial quantity for this new product: ")

        new_product_cost_price = 0.0
        while True:
            try:
                new_product_cost_price_str = input("Enter cost price per unit for the new product: ")
                new_product_cost_price = float(new_product_cost_price_str)
                if new_product_cost_price <= 0:
                    print("Cost price must be a positive number.")
            except ValueError:
                print("Invalid price format. Please enter a number (e.g., 150.75).")
            else:
                break

        new_product_origin = input("Enter new product's Country of Origin: ")

        new_id = 0

```

Figure 15

```

# main.py
import read
import write
import operations

# Global constants for company details and filenames
COMPANY_NAME = "TopskinCare Wholesale"
COMPANY_ADDRESS = "Madhyapur, Thimi, Bhaktapur"
PRODUCTS_FILENAME = "Products.txt"
BILL_COUNTER_FILENAME = "bill_counter.txt"

def display_welcome_message():
    """Prints the welcome message for the system."""
    print("\n" * 2)
    print("\t\t\t\t\t" + COMPANY_NAME)
    print("\n")
    print("\t\t\t\t\t" + COMPANY_ADDRESS)
    print("\n")

def main_program_loop():
    """Runs the main interactive loop for the wholesale system."""
    display_welcome_message()

    products_inventory = read.load_products(PRODUCTS_FILENAME)

    system_active = True
    while system_active:
        print("\n===== Main Menu =====")
        print("1. Sell Product(s) to Customer")
        print("2. Purchase Product(s) (Restock/Add New)")
        print("3. Display All Available Products")
        print("4. Exit System")
        print("=====")

        user_choice_str = input("Enter your choice (1-4): ")

        try:
            main_menu_choice = int(user_choice_str)
        except ValueError:
            print("Error: Invalid input! Please enter a number between 1 and 4.")
            continue
        except Exception as e:
            print("An unexpected error occurred with your input: " + str(e))
            continue

        if main_menu_choice == 1: # Sell Products
            updated_inventory_after_sale, sale_bill_details = operations.sell_products(products_inventory)

            if sale_bill_details:
                products_inventory = updated_inventory_after_sale

                current_bill_num = read.read_bill_counter(BILL_COUNTER_FILENAME)
                new_bill_num = current_bill_num + 1
                unique_bill_id = write.generate_unique_bill_id(new_bill_num)

                write.print_and_save_bill(
                    unique_bill_id,
                    sale_bill_details["customer_name"],
                    sale_bill_details["items_list"],
                    sale_bill_details["subtotal"],
                    sale_bill_details["discount_amount"],
                    sale_bill_details["vat_amount"],
                    sale_bill_details["grand_total"],
                    "SALE",
                    COMPANY_NAME,
                    COMPANY_ADDRESS
                )
                write.save_products(PRODUCTS_FILENAME, products_inventory)
                write.write_bill_counter(new_bill_num, BILL_COUNTER_FILENAME)

```

Figure 16 Code of MAIN.PY

```

        else:
            print("Sale operation did not complete or no items were sold. No bill generated.")

    elif main_menu_choice == 2: # Purchase Products
        updated_inventory_after_purchase, purchase_bill_details = operations.purchase_products(products_inventory)

        products_inventory = updated_inventory_after_purchase # Persist any changes even if no bill

        if purchase_bill_details:
            current_bill_num = read.read_bill_counter(BILL_COUNTER_FILENAME)
            new_bill_num = current_bill_num + 1
            unique_bill_id = write.generate_unique_bill_id(new_bill_num)

            write.print_and_save_bill(
                unique_bill_id,
                purchase_bill_details["customer_name"],
                purchase_bill_details["items_list"],
                purchase_bill_details["subtotal"],
                purchase_bill_details["discount_amount"], # Will be 0
                purchase_bill_details["vat_amount"], # Will be 0
                purchase_bill_details["grand_total"],
                "PURCHASE",
                COMPANY_NAME,
                COMPANY_ADDRESS
            )
            write.write_bill_counter(new_bill_num, BILL_COUNTER_FILENAME)
        else:
            print("Purchase operation did not complete with billable items or was cancelled.")

        write.save_products(PRODUCTS_FILENAME, products_inventory) # Save after any purchase op

    elif main_menu_choice == 3: # Display Products
        operations.display_products(products_inventory)

    elif main_menu_choice == 4: # Exit
        print("\nThank you for using the " + COMPANY_NAME + " system.")
        print("Exiting now...")
        system_active = False

    else:
        print("Error: Invalid choice. Please select an option from 1 to 4.")

__name__ == "__main__":
    main_program_loop()

```

Figure 17

4 TESTING

Show the purchase and sale of the product (Show the complete process)

```

                                TopskinCare Wholesale

                                Madhyapur, Thimi, Bhaktapur

===== Main Menu =====
1. Sell Product(s) to Customer
2. Purchase Product(s) (Restock/Add New)
3. Display All Available Products
4. Exit System
=====
Enter your choice (1-4): 1
Enter Customer Name: Suprem Thapa

----- AVAILABLE PRODUCTS -----
ID      Name                    Brand                    Qty      Price (Cost)      Origin
-----
1       Vitamin C Serum         Garnier                 351      1000.00           France
2       Skin Cleanser            Cetaphil               1153     280.00            Switzerland
3       Sunscreen                Aqualogica            500      700.00            India
4       FaceWash                 Himalaya               150      150.00            India
5       Moisturizer              Ponds                 230      200.00            USA
6       Sunscreen                Neutrogena            225      350.00            UK
7       cleanser                  cetaphil               100      650.00            canada
8       facecream                 suprem                 100      800.00            Nepal
9       ash                      ash                    700      700.00            nepal
-----
Enter Product ID to sell (or 0 to finish current sale): 2
Selected: Skin Cleanser (Stock: 1153, Selling Price/unit: Rs. 560.00)
Enter quantity to sell: 200
200 Skin Cleanser(s) added to bill.
BONUS: Customer receives 66 Skin Cleanser(s) for free!
Sell another product in this transaction? (yes/no): 1

--- Sale Transaction Summary ---
Skin Cleanser - Qty Paid: 200, Free: 66, Total Deducted: 266

```

Figure 18 Showing the running process of my code

TopskinCare Wholesale
Madhyapur, Thimi, Bhaktapur

----- SALE INVOICE -----

Bill ID: 20250514_9
Date: 2025-05-14 Time: 08:57:47

Customer Name: Suprem Thapa

S.N.	Product Name	Qty	Unit Price	Total Price
1	Skin Cleanser	200	560.00	112000.00
Subtotal:			Rs. 112000.00	
Net Amount:			Rs. 112000.00	
VAT (13%):			Rs. 14560.00	
GRAND TOTAL:			Rs. 126560.00	

Thank you for your purchase, Suprem Thapa!

Figure 19 invoice of sale

```

Bill successfully saved as SALE_BILL_20250514_9.txt

===== Main Menu =====
1. Sell Product(s) to Customer
2. Purchase Product(s) (Restock/Add New)
3. Display All Available Products
4. Exit System
=====
Enter your choice (1-4): 2

--- Purchase from Manufacturer/Supplier ---

Purchase Options:
1. Restock an existing product|
2. Add a completely new product
3. Finish current purchase transaction
Enter your choice (1-3): 1

----- AVAILABLE PRODUCTS -----
ID      Name                Brand                Qty      Price (Cost)      Origin
-----
1       Vitamin C Serum          Garnier              351      1000.00           France
2       Skin Cleanser            Cetaphil             887      280.00            Switzerland
3       Sunscreen                Aqualogica          500      700.00            India
4       FaceWash                 Himalaya             150      150.00            India
5       Moisturizer              Ponds                230      200.00            USA
6       Sunscreen                Neutrogena           225      350.00            UK
7       cleanser                 cetaphil             100      650.00            canada
8       facecream                suprem               100      800.00            Nepal
9       ash                      ash                  700      700.00            nepal
-----

Enter Product ID of existing product to restock: 4
Selected for restock: FaceWash (Current Stock: 150, Cost Price/unit: Rs. 150.00)
Enter quantity to add to stock: 200
200 FaceWash(s) restocked. New stock: 350

Purchase Options:
1. Restock an existing product
2. Add a completely new product
3. Finish current purchase transaction
Enter your choice (1-3): 2

--- Add New Product to Inventory ---
Enter new product's Name: arohancream
Enter new product's Brand: suprem
Enter initial quantity for this new product: 200
Enter cost price per unit for the new product: 349
Enter new product's Country of Origin: Nepal
New product 'arohancream' (ID: 10) added with stock 200.

Purchase Options:
1. Restock an existing product
2. Add a completely new product
3. Finish current purchase transaction
Enter your choice (1-3): 3

```

Figure 20 Selecting option 2

TopskinCare Wholesale
Madhyapur, Thimi, Bhaktapur

----- PURCHASE INVOICE -----

Bill ID: 20250514_10
Date: 2025-05-14 Time: 08:59:04

Supplier: Manufacturer/Internal

S.N.	Product Name	Qty	Unit Price	Total Price
1	FaceWash	200	150.00	30000.00
2	arohancream	200	349.00	69800.00
Subtotal:			Rs. 99800.00	
GRAND TOTAL:			Rs. 99800.00	

Items purchased/restocked successfully.

Figure 21 Invoice of restock product

```

Bill successfully saved as PURCHASE_BILL_20250514_10.txt

===== Main Menu =====
1. Sell Product(s) to Customer
2. Purchase Product(s) (Restock/Add New)
3. Display All Available Products
4. Exit System
=====
Enter your choice (1-4): 3

----- AVAILABLE PRODUCTS -----
ID      Name                Brand                Qty      Price (Cost)      Origin
-----
1       Vitamin C Serum          Garnier              351      1000.00           France
2       Skin Cleanser            Cetaphil             887      280.00            Switzerland
3       Sunscreen                Aqualogica          500      700.00            India
4       FaceWash                 Himalaya             350      150.00            India
5       Moisturizer              Ponds                230      200.00            USA
6       Sunscreen                Neutrogena           225      350.00            UK
7       cleanser                 cetaphil             100      650.00            canada
8       facecream                suprem               100      800.00            Nepal
9       ash                      ash                  700      700.00            nepal
10      arohancream              suprem               200      349.00            Nepal

===== Main Menu =====
1. Sell Product(s) to Customer
2. Purchase Product(s) (Restock/Add New)
3. Display All Available Products
4. Exit System
=====
Enter your choice (1-4): 4

Thank you for using the TopskinCare Wholesale system.
Exiting now...

```

Figure 22 Running option 3 Figure 23 Running option 4

```

Vitamin C Serum, Garnier, 351, 1000.00, France
Skin Cleanser, Cetaphil, 887, 280.00, Switzerland
Sunscreen, Aqualogica, 500, 700.00, India
FaceWash, Himalaya, 350, 150.00, India
Moisturizer, Ponds, 230, 200.00, USA
Sunscreen, Neutrogena, 225, 350.00, UK
cleanser, cetaphil, 100, 650.00, canada
facecream, suprem, 100, 800.00, Nepal
ash, ash, 700, 700.00, nepal
arohancream, suprem, 200, 349.00, Nepal

```

Figure 24 Txt file of product

Opening text and show the bill

TopskinCare Wholesale
Madhyapur, Thimi, Bhaktapur

----- SALE INVOICE -----

Bill ID: 20250514_9
Date: 2025-05-14 Time: 08:57:47

Customer Name: Suprem Thapa

S.N.	Product Name	Qty	Unit Price	Total Price
1	Skin Cleanser	200	560.00	112000.00
Subtotal:			Rs. 112000.00	
Net Amount:			Rs. 112000.00	
VAT (13%):			Rs. 14560.00	
GRAND TOTAL:			Rs. 126560.00	

Thank you for your purchase, Suprem Thapa!

Figure 25 Invoice after sale

TopskinCare Wholesale
Madhyapur, Thimi, Bhaktapur

----- PURCHASE INVOICE -----

Bill ID: 20250514_10

Date: 2025-05-14 Time: 08:59:04

Supplier: Manufacturer/Internal

S.N.	Product Name	Qty	Unit Price	Total Price
1	FaceWash	200	150.00	30000.00
2	arohancream	200	349.00	69800.00
Subtotal:			Rs. 99800.00	
GRAND TOTAL:			Rs. 99800.00	

Items purchased/restocked successfully.

Figure 26 Invoice in txt after restock

Also at the end show the termination of the program as well after selecting an option

```

Bill successfully saved as PURCHASE_BILL_20250514_10.txt

===== Main Menu =====
1. Sell Product(s) to Customer
2. Purchase Product(s) (Restock/Add New)
3. Display All Available Products|
4. Exit System
=====
Enter your choice (1-4): 3

----- AVAILABLE PRODUCTS -----
ID      Name                Brand                Qty      Price (Cost)      Origin
-----
1       Vitamin C Serum          Garnier              351      1000.00           France
2       Skin Cleanser             Cetaphil             887      280.00            Switzerland
3       Sunscreen                 Aqualogica          500      700.00            India
4       FaceWash                  Himalaya             350      150.00            India
5       Moisturizer               Ponds                230      200.00            USA
6       Sunscreen                 Neutrogena           225      350.00            UK
7       cleanser                  cetaphil             100      650.00            canada
8       facecream                 suprem               100      800.00            Nepal
9       ash                       ash                  700      700.00            nepal
10      arohancream               suprem               200      349.00            Nepal
-----

===== Main Menu =====
1. Sell Product(s) to Customer
2. Purchase Product(s) (Restock/Add New)
3. Display All Available Products
4. Exit System
=====
Enter your choice (1-4): 4

Thank you for using the TopskinCare Wholesale system.
Exiting now...

```

Figure 27 Running option 4

4.1 TEST 1

Objective	To verify that the system gracefully handles invalid input types for numerical fields using <code>try-except</code> blocks, preventing crashes and providing user-friendly error messages.
Action	Run the main.py script.
Expected Result	For each invalid non-numeric input, the system should display an error message indicating that a number is required (e.g., "Error: Invalid input! Please enter a number...", "Error: Invalid input. Please enter a numerical Product ID.", "Error: Invalid input. Please enter a numerical quantity."). The system should re-prompt the user for the correct input without crashing.
Actual Result	Show implementation of try, except, Provide invalid input and show the message
Conclusion	The system successfully implements <code>try-except ValueError</code> blocks for handling nonmessages and displayed the expected revalidation for data types. demonstrating robust

Table 1

Showing implementation of try, except

```

TopskinCare Wholesale

Madhyapur, Thimi, Bhaktapur

===== Main Menu =====
1. Sell Product(s) to Customer
2. Purchase Product(s) (Restock/Add New)
3. Display All Available Products
4. Exit System
=====
Enter your choice (1-4): a
Error: Invalid input! Please enter a number between 1 and 4.

```

Figure 28 finding invalid. input

4.2 TEST 2

Objective	To verify that the system correctly handles invalid product ID selections (negative values, non-existent IDs) and invalid quantity inputs (negative values) during sale and purchase operations.
Action	Run <code>main.py</code> . a "Sell Product(s)" operation (Choice 1) or "Purchase Choice 2, then "Restock")
Expected Result	The system should not crash and should allow the user to correct their input.

Actual Result	Console Output for Non-Existent Product ID Console Output for Negative Quantity
Conclusion	The system effectively validates Product ID and quantity inputs. It correctly identified and reported errors for negative values, nonexistent Product IDs, and non-positive quantities, re-prompting the user as expected

Table 2

Selection purchase and sale of products

- Provide the negative value as input
- Provide the non existed value as input

```

===== Main Menu =====
1. Sell Product(s) to Customer
2. Purchase Product(s) (Restock/Add New)
3. Display All Available Products
4. Exit System
=====
Enter your choice (1-4): -1
Error: Invalid choice. Please select an option from 1 to 4.

===== Main Menu =====
1. Sell Product(s) to Customer
2. Purchase Product(s) (Restock/Add New)
3. Display All Available Products
4. Exit System
=====
Enter your choice (1-4): 5
Error: Invalid choice. Please select an option from 1 to 4.

```

Figure 29 Provide the negative value as input, Provide the non existed value as input

4.3 TEST 3

Objective	To verify the complete purchase process for multiple products (restocking existing and adding new), including correct stock updates, bill calculation (no VAT, no discount), console output, and bill file generation.
Action	Select "Purchase Product(s)" (Choice 2)
Expected Result	The content of this file should match the console bill output. Products.txt should be updated: the restocked product's quantity should increase, and the new product should be added.
	bill_counter.txt should be incremented.
Actual Result	<i>The content of bill_counter.txt before and after.)</i>
Conclusion	The purchase process for multiple items (restock and new) functioned correctly. Stock levels in Products.txt were updated as expected, the bill was calculated without VAT or discount, and the purchase bill was accurately generated both on the console and in its dedicated text file. The bill counter was also updated.

Table 3

File generation of purchase of product(s) (Purchasing multiple product(s))

Show complete purchase process

```

                                TopskinCare Wholesale

                                Madhyapur, Thimi, Bhaktapur

===== Main Menu =====
1. Sell Product(s) to Customer
2. Purchase Product(s) (Restock/Add New)
3. Display All Available Products
4. Exit System
=====
Enter your choice (1-4): 2

--- Purchase from Manufacturer/Supplier ---

Purchase Options:
1. Restock an existing product
2. Add a completely new product
3. Finish current purchase transaction
Enter your choice (1-3): 0
Invalid choice. Please enter 1, 2, or 3.

Purchase Options:
1. Restock an existing product
2. Add a completely new product
3. Finish current purchase transaction
Enter your choice (1-3): 1

----- AVAILABLE PRODUCTS -----
ID      Name                      Brand                      Qty      Price (Cost)      Origin
-----
1       Vitamin C Serum                Garnier                   335      1000.00           France
2       Skin Cleanser                  Cetaphil                  887      280.00            Switzerland
3       Sunscreen                      Aqualogica               512      700.00            India
4       FaceWash                      Himalaya                  350      150.00            India
5       Moisturizer                   Ponds                    230      200.00            USA
6       Sunscreen                      Neutrogena               225      350.00            UK
7       cleanser                      cetaphil                  100      650.00            canada
8       facecream                     suprem                    100      800.00            Nepal
9       ash                           ash                       700      700.00            nepal
10      arohancream                    suprem                    200      349.00            Nepal

Enter Product ID of existing product to restock: 11
Error: Product ID not found. Please enter a valid ID from the list.
Enter Product ID of existing product to restock: a
Error: Invalid input. Please enter a numerical Product ID.
Enter Product ID of existing product to restock: 1
Selected for restock: Vitamin C Serum (Current Stock: 335, Cost Price/unit: Rs. 1000.00)
Enter quantity to add to stock: 30
30 Vitamin C Serum(s) restocked. New stock: 365

```

Figure 30 Show complete purchase process and output in shell

```

Purchase Options:
1. Restock an existing product
2. Add a completely new product
3. Finish current purchase transaction
Enter your choice (1-3): 2

--- Add New Product to Inventory ---
Enter new product's Name: lip balm
Enter new product's Brand: garnier
Enter initial quantity for this new product: 700
Enter cost price per unit for the new product: 250
Enter new product's Country of Origin: nepal
New product 'lip balm' (ID: 11) added with stock 700.

Purchase Options:
1. Restock an existing product
2. Add a completely new product
3. Finish current purchase transaction
Enter your choice (1-3): 3

                                TopskinCare Wholesale
                                Madhyapur, Thimi, Bhaktapur

----- PURCHASE INVOICE -----
Bill ID: 20250514_13
Date: 2025-05-14      Time: 10:17:16

Supplier: Manufacturer/Internal

-----
S.N.      Product Name      Qty      Unit Price      Total Price
-----
1          Vitamin C Serum    30        1000.00        30000.00
2          lip balm           700       250.00        175000.00
-----
Subtotal:                Rs. 205000.00
-----
GRAND TOTAL:             Rs. 205000.00
-----

Items purchased/restocked successfully.

```

Figure 31 Finally show the purchased products details in a text file

4.4 TEST TEST 4

Objective	To verify the complete sales process for multiple products, including correct stock deduction (with "buy 3, get 1 free"), application of the 10% clearance discount if a product goes out of stock, 13% VAT calculation on the net amount, console output, and bill file generation.
Action	Select "Sell Product(s) to Customer" (Choice 1). Enter a customer name
Expected Result	Products.txt should be updated with correctly deducted quantities (including free). t should be incremented. bill_counter.tx
Actual Result	<i>The console showing the entire sales process (prompts, inputs, item confirmations, free item messages, out-of-stock info message if applicable).contain of bill and produt fill efore and after</i>
Conclusion	The sales process for multiple items functioned as expected. Stock deductions, including the "buy 3, get 1 free" logic, were accurate. The conditional 10% clearance discount was correctly applied when a product went out of stock, and the 13% VAT was calculated

	on the resulting net amount. The sales bill was accurately generated on the console and in the text file, and the product inventory and bill counter were updated correctly.
--	--

Table 4

Bill successfully saved as PURCHASE_BILL_20250514_13.txt

===== Main Menu =====

1. Sell Product(s) to Customer
2. Purchase Product(s) (Restock/Add New)
3. Display All Available Products
4. Exit System

=====

Enter your choice (1-4): 1

Enter Customer Name: ash

----- AVAILABLE PRODUCTS -----					
ID	Name	Brand	Qty	Price (Cost)	Origin
1	Vitamin C Serum	Garnier	365	1000.00	France
2	Skin Cleanser	Cetaphil	887	280.00	Switzerland
3	Sunscreen	Aqualogica	512	700.00	India
4	FaceWash	Himalaya	350	150.00	India
5	Moisturizer	Ponds	230	200.00	USA
6	Sunscreen	Neutrogena	225	350.00	UK
7	cleanser	cetaphil	100	650.00	canada
8	facecream	suprem	100	800.00	Nepal
9	ash	ash	700	700.00	nepal
10	arohancream	suprem	200	349.00	Nepal
11	lip balm	garnier	700	250.00	nepal

Enter Product ID to sell (or 0 to finish current sale): 12

Error: Product ID not found. Please enter a valid ID from the list.

Enter Product ID to sell (or 0 to finish current sale): 1

Selected: Vitamin C Serum (Stock: 365, Selling Price/unit: Rs. 2000.00)

Enter quantity to sell: 60

60 Vitamin C Serum(s) added to bill.

BONUS: Customer receives 20 Vitamin C Serum(s) for free!

Sell another product in this transaction? (yes/no): yes

----- AVAILABLE PRODUCTS -----					
ID	Name	Brand	Qty	Price (Cost)	Origin
1	Vitamin C Serum	Garnier	285	1000.00	France
2	Skin Cleanser	Cetaphil	887	280.00	Switzerland
3	Sunscreen	Aqualogica	512	700.00	India
4	FaceWash	Himalaya	350	150.00	India
5	Moisturizer	Ponds	230	200.00	USA
6	Sunscreen	Neutrogena	225	350.00	UK
7	cleanser	cetaphil	100	650.00	canada
8	facecream	suprem	100	800.00	Nepal
9	ash	ash	700	700.00	nepal
10	arohancream	suprem	200	349.00	Nepal
11	lip balm	garnier	700	250.00	nepal

Enter Product ID to sell (or 0 to finish current sale): 2

Selected: Skin Cleanser (Stock: 887, Selling Price/unit: Rs. 560.00)

Enter quantity to sell: a

Error: Invalid input. Please enter a numerical quantity.

Enter quantity to sell: 20

20 Skin Cleanser(s) added to bill.

BONUS: Customer receives 6 Skin Cleanser(s) for free!

Sell another product in this transaction? (yes/no): yes

Figure 32 Show the complete sales process of the product(s) Show output in the shell as well.

ID	Name	Brand	Qty	Price (Cost)	Origin
1	Vitamin C Serum	Garnier	285	1000.00	France
2	Skin Cleanser	Cetaphil	861	280.00	Switzerland
3	Sunscreen	Aqualogica	512	700.00	India
4	FaceWash	Himalaya	350	150.00	India
5	Moisturizer	Ponds	230	200.00	USA
6	Sunscreen	Neutrogena	225	350.00	UK
7	cleanser	cetaphil	100	650.00	canada
8	facecream	suprem	100	800.00	Nepal
9	ash	ash	700	700.00	nepal
10	arohancream	suprem	200	349.00	Nepal
11	lip balm	garnier	700	250.00	nepal

Enter Product ID to sell (or 0 to finish current sale): 3
 Selected: Sunscreen (Stock: 512, Selling Price/unit: Rs. 1400.00)
 Enter quantity to sell: 500
 Error: Not enough stock for Sunscreen.
 Available: 512, Required (incl. free): 666. Please try a smaller quantity.

----- AVAILABLE PRODUCTS -----					
ID	Name	Brand	Qty	Price (Cost)	Origin
1	Vitamin C Serum	Garnier	285	1000.00	France
2	Skin Cleanser	Cetaphil	861	280.00	Switzerland
3	Sunscreen	Aqualogica	512	700.00	India
4	FaceWash	Himalaya	350	150.00	India
5	Moisturizer	Ponds	230	200.00	USA
6	Sunscreen	Neutrogena	225	350.00	UK
7	cleanser	cetaphil	100	650.00	canada
8	facecream	suprem	100	800.00	Nepal
9	ash	ash	700	700.00	nepal
10	arohancream	suprem	200	349.00	Nepal
11	lip balm	garnier	700	250.00	nepal

Enter Product ID to sell (or 0 to finish current sale): 0

--- Sale Transaction Summary ---
 Vitamin C Serum - Qty Paid: 60, Free: 20, Total Deducted: 80
 Skin Cleanser - Qty Paid: 20, Free: 6, Total Deducted: 26

TopskinCare Wholesale
 Madhyapur, Thimi, Bhaktapur

----- SALE INVOICE -----
 Bill ID: 20250514_14
 Date: 2025-05-14 Time: 10:21:57

Customer Name: ash

S.N.	Product Name	Qty	Unit Price	Total Price
1	Vitamin C Serum	60	2000.00	120000.00
2	Skin Cleanser	20	560.00	11200.00
		Subtotal:	Rs. 131200.00	
		Net Amount:	Rs. 131200.00	
		VAT (13%):	Rs. 17056.00	
		GRAND TOTAL:	Rs. 148256.00	

Thank you for your purchase, ash!

Figure 33 Finally show the sold product(s) details in text file

4.5 TEST 5

Objective	To explicitly verify that product stock quantities are correctly updated in the Products.txt file after both selling and purchasing operations.
Action	Selling Operation, Verification after Selling, Purchasing Operation (Restock), Verification after Purchasing
Expected Result	After selling The quantity of the sold product in Products.txt should be reduced by the requested_quantity plus free_items given. any e restocked nity_to_add After. Purchasing(Restock) in Products.txt should be increased by the
Actual Result	As in given screen shot
Conclusion	The system accurately updates the stock quantities in the Products.txt file. The quantity was correctly decremented during sales (accounting for any promotional free items) and correctly incremented during purchase/restock operations, as verified by comparing the file contents before and after each transaction type

Table 5

Bill successfully saved as PURCHASE_BILL_20250514_13.txt

===== Main Menu =====

1. Sell Product(s) to Customer
2. Purchase Product(s) (Restock/Add New)
3. Display All Available Products
4. Exit System

=====

Enter your choice (1-4): 1

Enter Customer Name: ash

----- AVAILABLE PRODUCTS -----					
ID	Name	Brand	Qty	Price (Cost)	Origin
1	Vitamin C Serum	Garnier	365	1000.00	France
2	Skin Cleanser	Cetaphil	887	280.00	Switzerland
3	Sunscreen	Aqualogica	512	700.00	India
4	FaceWash	Himalaya	350	150.00	India
5	Moisturizer	Ponds	230	200.00	USA
6	Sunscreen	Neutrogena	225	350.00	UK
7	cleanser	cetaphil	100	650.00	canada
8	facecream	suprem	100	800.00	Nepal
9	ash	ash	700	700.00	nepal
10	arohancream	suprem	200	349.00	Nepal
11	lip balm	garnier	700	250.00	nepal

Enter Product ID to sell (or 0 to finish current sale): 12

Error: Product ID not found. Please enter a valid ID from the list.

Enter Product ID to sell (or 0 to finish current sale): 1

Selected: Vitamin C Serum (Stock: 365, Selling Price/unit: Rs. 2000.00)

Enter quantity to sell: 60

60 Vitamin C Serum(s) added to bill.

BONUS: Customer receives 20 Vitamin C Serum(s) for free!

Sell another product in this transaction? (yes/no): yes

----- AVAILABLE PRODUCTS -----					
ID	Name	Brand	Qty	Price (Cost)	Origin
1	Vitamin C Serum	Garnier	285	1000.00	France
2	Skin Cleanser	Cetaphil	887	280.00	Switzerland
3	Sunscreen	Aqualogica	512	700.00	India
4	FaceWash	Himalaya	350	150.00	India
5	Moisturizer	Ponds	230	200.00	USA
6	Sunscreen	Neutrogena	225	350.00	UK
7	cleanser	cetaphil	100	650.00	canada
8	facecream	suprem	100	800.00	Nepal
9	ash	ash	700	700.00	nepal
10	arohancream	suprem	200	349.00	Nepal
11	lip balm	garnier	700	250.00	nepal

Enter Product ID to sell (or 0 to finish current sale): 2

Selected: Skin Cleanser (Stock: 887, Selling Price/unit: Rs. 560.00)

Enter quantity to sell: a

Error: Invalid input. Please enter a numerical quantity.

Enter quantity to sell: 20

20 Skin Cleanser(s) added to bill.

BONUS: Customer receives 6 Skin Cleanser(s) for free!

Sell another product in this transaction? (yes/no): yes

Figure 34 Show the update in stock of product(s)

```

TopskinCare Wholesale

Madhyapur, Thimi, Bhaktapur

===== Main Menu =====
1. Sell Product(s) to Customer
2. Purchase Product(s) (Restock/Add New)
3. Display All Available Products
4. Exit System
=====
Enter your choice (1-4): 2

--- Purchase from Manufacturer/Supplier ---

Purchase Options:
1. Restock an existing product
2. Add a completely new product
3. Finish current purchase transaction
Enter your choice (1-3): 1

----- AVAILABLE PRODUCTS -----
ID      Name                Brand                Qty      Price (Cost)      Origin
-----
1       Vitamin C Serum          Garnier              285      1000.00           France
2       Skin Cleanser            Cetaphil             861      280.00            Switzerland
3       Sunscreen                Aqualogica          512      700.00            India
4       FaceWash                 Himalaya             350      150.00            India
5       Moisturizer              Ponds               230      200.00            USA
6       Sunscreen                Neutrogena          225      350.00            UK
7       cleanser                 cetaphil             100      650.00            canada
8       facecream                suprem               100      800.00            Nepal
9       ash                       ash                  700      700.00            nepal
10      arohancream              suprem               200      349.00            Nepal
11      lip balm                  garnier              700      250.00            nepal
-----

Enter Product ID of existing product to restock: 11
Selected for restock: lip balm (Current Stock: 700, Cost Price/unit: Rs. 250.00)
Enter quantity to add to stock: 300
300 lip balm(s) restocked. New stock: 1000

Purchase Options:
1. Restock an existing product
2. Add a completely new product
3. Finish current purchase transaction
Enter your choice (1-3): 3

```

Figure 35 Show the update in stock of quantity

```

                                TopskinCare Wholesale

                                Madhyapur, Thimi, Bhaktapur

===== Main Menu =====
1. Sell Product(s) to Customer
2. Purchase Product(s) (Restock/Add New)
3. Display All Available Products
4. Exit System
=====
Enter your choice (1-4): 3

----- AVAILABLE PRODUCTS -----
ID      Name                Brand                Qty      Price (Cost)      Origin
-----
1       Vitamin C Serum      Garnier              285      1000.00           France
2       Skin Cleanser         Cetaphil             861      280.00            Switzerland
3       Sunscreen              Aqualogica          512      700.00            India
4       FaceWash               Himalaya             350      150.00            India
5       Moisturizer            Ponds                230      200.00            USA
6       Sunscreen              Neutrogena           225      350.00            UK
7       cleanser                cetaphil             100      650.00            canada
8       facecream              suprem               100      800.00            Nepal
9       ash                    ash                  700      700.00            nepal
10      arohancream              suprem               200      349.00            Nepal
11      lip balm                 garnier              1000     250.00            nepal
-----

===== Main Menu =====
1. Sell Product(s) to Customer
2. Purchase Product(s) (Restock/Add New)
3. Display All Available Products
4. Exit System
=====
Enter your choice (1-4): |

```

Figure 36 Show the update in stock of quantity

Vitamin C Serum, Garnier,285,1000.00, France
 Skin Cleanser, Cetaphil,861,280.00, Switzerland
 Sunscreen, Aqualogica,512,700.00, India
 FaceWash,Himalaya,350,150.00,India
 Moisturizer,Ponds,230,200.00,USA
 Sunscreen,Neutrogena,225,350.00,UK
 cleanser,cetaphil,100,650.00,canada
 facecream,suprem,100,800.00,Nepal
 ash,ash,700,700.00,nepal
 arohancream,suprem,200,349.00,Nepal
 lip balm,garnier,700,250.00,nepal

Figure 37 Show the quantity being deducted while purchasing the product

Vitamin C Serum, Garnier,285,1000.00, France
 Skin Cleanser, Cetaphil,861,280.00, Switzerland
 Sunscreen, Aqualogica,512,700.00, India
 FaceWash,Himalaya,350,150.00,India
 Moisturizer,Ponds,230,200.00,USA
 Sunscreen,Neutrogena,225,350.00,UK
 cleanser,cetaphil,100,650.00,canada
 facecream,suprem,100,800.00,Nepal
 ash,ash,700,700.00,nepal
 arohancream,suprem,200,349.00,Nepal
 lip balm,garnier,1000,250.00,nepal

Figure 38 Show the quantity being added while purchasing the product



Vitamin C Serum, Garnier, ~~365~~, 1000.00, France
 Skin Cleanser, Cetaphil, 861, 280.00, Switzerland
 Sunscreen, Aqualogica, 512, 700.00, India
 FaceWash, Himalaya, 350, 150.00, India
 Moisturizer, Ponds, 230, 200.00, USA
 Sunscreen, Neutrogena, 225, 350.00, UK
 cleanser, cetaphil, 100, 650.00, canada
 facecream, suprem, 100, 800.00, Nepal
 ash, ash, 700, 700.00, nepal
 arohancream, suprem, 200, 349.00, Nepal
 lip balm, garnier, 1000, 250.00, nepal

Figure 39 Show the quantity being added while selling the product



Vitamin C Serum, Garnier, ~~285~~, 1000.00, France
 Skin Cleanser, Cetaphil, 861, 280.00, Switzerland
 Sunscreen, Aqualogica, 512, 700.00, India
 FaceWash, Himalaya, 350, 150.00, India
 Moisturizer, Ponds, 230, 200.00, USA
 Sunscreen, Neutrogena, 225, 350.00, UK
 cleanser, cetaphil, 100, 650.00, canada
 facecream, suprem, 100, 800.00, Nepal
 ash, ash, 700, 700.00, nepal
 arohancream, suprem, 200, 349.00, Nepal
 lip balm, garnier, 1000, 250.00, nepal

Figure 40 Show the quantity being deducting while selling the product

5 CONCLUSION

The TopskinCare Wholesale system successfully implements a foundational commandline interface for managing product inventory, processing sales, and handling purchases. Through a modular design encompassing `read.py` for data loading, `write.py` for data persistence and bill generation, `operations.py` for core business logic, and `main.py` for user interaction, the system provides essential functionalities. Key features include dynamic product management with stock updates, automated "buy 3, get 1 free" sales promotions, and comprehensive bill generation for both sales and purchase transactions, complete with VAT and discount calculations. The consistent use of basic Python data structures like dictionaries and lists, coupled with file-based data storage, ensures simplicity and adherence to fundamental programming principles.

A critical aspect of the system's design is its focus on user experience within a text-based environment, offering clear menus, input validation, and informative feedback. The sequential generation of product IDs and unique bill IDs, incorporating date and a counter, provides a reliable method for tracking transactions and inventory changes. While the current implementation relies on manual formatting and sorting algorithms to meet specific constraints, it demonstrates a solid understanding of procedural programming and data handling without relying on more advanced built-in functions or external libraries, showcasing resourcefulness in problem-solving.

Looking forward, this system provides a robust baseline for future enhancements. Potential improvements could include more sophisticated error handling, advanced search and filtering capabilities for products, user authentication for security, and perhaps a transition to a more persistent database solution for scalability. Furthermore, introducing features like sales reporting, low-stock alerts, and integration with a graphical user interface could significantly expand its utility. Nevertheless, as it stands, the TopskinCare Wholesale

system effectively meets its initial objectives of providing a functional and maintainable inventory and billing solution using core Python programming.

6 APPENDIX

read.py

```
def load_products(filename="Products.txt"):
```

```
    """
```

```
    Loads product data from a specified file.
```

```
    Each line in the file should be: name,brand,quantity,price,origin
```

```
    Returns a dictionary of products.
```

```
    """
```

```
    products_data = {}    try:
```

```
    with open(filename, "r") as file:
```

```
        product_id_counter = 1
```

```
    for line in file:
```

```
        # Manually remove newline character if present at the end of the line
```

```
    if line and line[-1] == '\n': # Check if line is not empty before accessing -1
```

```
    line = line[:-1]
```

```
        if not line: # Skip empty lines
```

```
        continue
```

```
        parts =
```

```
        line.split(",")                      if
```

```
        len(parts) == 5:
```

```
        name = parts[0]
```

```
        brand = parts[1]
```

```
        try:
```

```

        quantity = int(parts[2])
price = float(parts[3])          except
ValueError:
        # Using string concatenation as f-strings/format are disallowed
print("Warning: Invalid quantity or price for product data: " + line + ". Skipping.")
        continue          origin = parts[4]
products_data[product_id_counter] = [name, brand, quantity, price, origin]
product_id_counter += 1          else:
        print("Warning: Malformed line in " + filename + ": " + line + ". Skipping.")
except FileNotFoundError:
        print("Error: " + filename + " not found. Starting with an empty inventory.")
except Exception as e:
        # Using str(e) for error message          print("An unexpected error
occurred while loading products: " + str(e))          return products_data

def read_bill_counter(filename="bill_counter.txt"):
    """
    Reads the current bill counter value from its file.
    Returns the counter value, or 0 if file not found or invalid.
    """
    try:          with
open(filename, "r") as file:
counter_line = file.readline()
        # Remove newline if present, as int() can fail with it. No strip() allowed.
if counter_line and counter_line[-1] == '\n':
        counter_line = counter_line[:-1]

        if not counter_line: # Handle empty file or line
            print("Warning: " + filename + " is empty. Resetting counter to 0.")
return 0

```

```

        counter = int(counter_line)
return counter    except
FileNotFoundError:
        return 0 # Start from 0 if file doesn't exist
except ValueError:
        print("Warning: " + filename + " contains invalid data. Resetting counter to 0.")
return 0    except Exception as e:
        print("Error reading bill counter from " + filename + ": " + str(e))
return 0

```

FOR Write.py # write.py import datetime # Allowed for
datetime.datetime.now() parts, not strftime

```

def save_products(filename, products_data):
    """
    Saves the current product data back to the specified file.
    """
    try:
        with open(filename, "w") as file:
            product_ids_to_write = list(products_data.keys())
            n = len(product_ids_to_write)
            for i in range(n):
                for j in range(0, n-i-1):
                    # Ensure keys are comparable, assuming they are integers
                    if int(product_ids_to_write[j]) > int(product_ids_to_write[j+1]):
                        product_ids_to_write[j], product_ids_to_write[j+1] =
                        product_ids_to_write[j+1], product_ids_to_write[j]

            for product_id in product_ids_to_write:
                details = products_data[product_id]
                line_parts = [
                    str(details[0]), # name
                    str(details[1]), # brand
                    str(details[2]), # quantity
                    "%.2f" % details[3], # price
                    str(details[4]) # origin
                ]

```

```

    ]
file.write(",".join(line_parts) + "\n")    except
Exception as e:
    print("Error saving products to " + filename + ": " + str(e))

def write_bill_counter(new_counter, filename="bill_counter.txt"):
    """
    Writes the new bill counter value to its file.
    """
    try:
        with open(filename,
"w") as file:
file.write(str(new_counter))
    except Exception as e:
        print("Error writing to " + filename + ": " + str(e))

```

```

def generate_unique_bill_id(counter):
    """
    Generates a unique bill ID using current date/time components and a counter.
    No strftime is used.
    """

    now = datetime.datetime.now()
year = str(now.year)

    month_val = now.month
month_str = str(month_val)    if
month_val < 10:
    month_str = "0" + month_str

    day_val = now.day
day_str = str(day_val)    if
day_val < 10:
    day_str = "0" + day_str

```

```
return year + month_str + day_str + "_" + str(counter)
```

```
def print_and_save_bill(bill_id, customer_name, items_list,
    subtotal, discount_amount, vat_amount, grand_total,
    bill_type, company_name, company_address):
```

```
    """
```

```
    Formats, prints to console, and saves a bill to a unique file.
```

```
    bill_type can be "SALE" or "PURCHASE".
```

```
    items_list is a list of dictionaries, each:
```

```
    {'name': str, 'quantity': int, 'price_per_unit': float, 'total': float}
```

```
    """
```

```
    bill_content_list = []
```

```
        bill_content_list.append("\n" * 2)
```

```
        bill_content_list.append("\t\t\t\t" + company_name + "\n")
```

```
        bill_content_list.append("\t\t\t\t" + company_address + "\n")
```

```
        bill_content_list.append("\n")
```

```
        if bill_type == "SALE":
```

```
            bill_content_list.append("----- SALE INVOICE ----- \n")
```

```
        elif bill_type == "PURCHASE":
```

```
            bill_content_list.append("----- PURCHASE INVOICE ----- \n")
```

```
        now = datetime.datetime.now()    month_val = now.month    month_str = str(month_val)
```

```
        if month_val < 10: month_str = "0" + month_str    day_val = now.day    day_str =
str(day_val)    if day_val < 10: day_str = "0" + day_str    date_str = str(now.year) + "-" +
month_str + "-" + day_str
```

```
        hour_val = now.hour    hour_str = str(hour_val)    if
```

```
hour_val < 10: hour_str = "0" + hour_str    minute_val =
```

```
now.minute    minute_str = str(minute_val)    if
```

```
minute_val < 10: minute_str = "0" + minute_str
```

```
second_val = now.second    second_str =
```

```
SUPREM THAPA
```

```

str(second_val)    if second_val < 10: second_str = "0" +
second_str    time_str = hour_str + ":" + minute_str + ":" +
second_str

```

```

    bill_content_list.append("Bill ID: " + bill_id + "\n")
bill_content_list.append("Date: " + date_str + "\tTime: " + time_str + "\n\n")

```

```

    if bill_type == "SALE":
        bill_content_list.append("Customer Name: " + customer_name + "\n\n")
    elif bill_type == "PURCHASE":
        bill_content_list.append("Supplier: Manufacturer/Internal\n\n")

```

```

    header_line = "S.N.\tProduct Name          \tQty\tUnit Price\tTotal Price\n"
    separator_line = "-" * 65 + "\n"

```

```

    bill_content_list.append(separator_line)
bill_content_list.append(header_line)    bill_content_list.append(separator_line)

```

```

    item_serial_number = 1    for item in
items_list:        name_col =
item['name']        max_name_len = 20
if len(name_col) > max_name_len:
    name_col = name_col[:max_name_len-3] + "..."
else:
    name_col = name_col + " " * (max_name_len - len(name_col))

```

```

    unit_price_str = "%.2f" % item['price_per_unit']
item_total_str = "%.2f" % item['total']

```

```

    line = str(item_serial_number) + "\t" + \
name_col + "\t" + \            str(item['quantity'])
+ "\t" + \            unit_price_str + "\t\t" + \

```

```

item_total_str + "\n"
bill_content_list.append(line)
item_serial_number += 1

    bill_content_list.append(separator_line)

    if bill_type == "SALE":
        bill_content_list.append("\t\t\t\tSubtotal: \tRs. " + ("%0.2f" % subtotal) + "\n")
        if discount_amount > 0: # Only print discount if it's applied
            bill_content_list.append("\t\t\t\tClearance Discount (10%): Rs. " + ("%0.2f" %
            discount_amount) + "\n")
            net_amount_for_display =
            subtotal - discount_amount
            bill_content_list.append("\t\t\t\tNet Amount:
            \tRs. " + ("%0.2f" %
            net_amount_for_display) + "\n")
            bill_content_list.append("\t\t\t\tVAT (13%): \tRs. " + ("%0.2f" % vat_amount) + "\n")
        elif bill_type == "PURCHASE":
            bill_content_list.append("\t\t\t\tSubtotal: \tRs. " + ("%0.2f" % subtotal) + "\n")
            # No Discount or VAT lines for purchase from manufacturer

            bill_content_list.append(separator_line)
            bill_content_list.append("\t\t\t\tGRAND
            TOTAL: \tRs. " + ("%0.2f" % grand_total) + "\n")
            bill_content_list.append(separator_line)

            if bill_type == "SALE":
                bill_content_list.append("\nThank you for your purchase, " + customer_name + "!\n")
            elif bill_type == "PURCHASE":
                bill_content_list.append("\nItems
                purchased/restocked successfully.\n")
                bill_content_list.append("\n" * 2)

            final_bill_string = "".join(bill_content_list)
            print(final_bill_string)

```

```
# operations.py
```

■■■■■

■■■■■

```
print(header)    print("-" * (len(header) + 15))
```

```
print("-----") return
```

ids to display[i]

```

    for product_id in ids_to_display:
        details = products_data[product_id]
    name = details[0]        brand =
    details[1]        quantity = details[2]

        cost_price = details[3]
    origin = details[4]

    name_col_width = 22
    brand_col_width = 22

    name_display = name        if
    len(name) > name_col_width:
        name_display = name[:name_col_width-3] + "..."
    else:
        name_display = name + " " * (name_col_width - len(name))

    brand_display = brand        if
    len(brand) > brand_col_width:
        brand_display = brand[:brand_col_width-3] + "..."
    else:
        brand_display = brand + " " * (brand_col_width - len(brand))

    price_str = "%.2f" % cost_price

    print(str(product_id) + "\t" + name_display + "\t" + brand_display + "\t" + \
    str(quantity) + "\t" + price_str + "\t\t" + origin)    print("-----
    -----")

def get_valid_product_id(products_data, prompt_message, allow_zero=False):
    SUPREM THAPA

```

```
"""
```

```
    Gets a valid product ID from user input.
```

```
    If allow_zero is True, 0 is considered a valid input (e.g., to stop adding items).
```

```
"""
```

```
    while True:
```

```
        try:
```

```
            pid_str =
```

```
            input(prompt_me
```

```
            ssage)
```

```
            pid = int(pid_str)
```

```
            if allow_zero and
```

```
            pid == 0:
```

```
                return 0            if pid not in products_data: # Assumes keys are
```

```
            integers            print("Error: Product ID not found. Please enter a valid ID from
```

```
            the list.")            else:
```

```
                return pid
```

```
        except ValueError:
```

```
            print("Error: Invalid input. Please enter a numerical Product ID.")
```

```
        except Exception as e:
```

```
            print("An unexpected error occurred with product ID input: " + str(e))
```

```
def get_valid_quantity(prompt_message):
```

```
    """
```

```
    Gets a valid positive integer quantity from user input.
```

```
    """
```

```
    while True:
```

```
        try:
```

```
            qty_str = input(prompt_message)
```

```
            qty = int(qty_str)            if qty <= 0:
```

```
                print("Error: Quantity must be a positive whole number.")
```

```
        else:
```

```
    SUPREM THAPA
```

```

        return qty
except ValueError:
    print("Error: Invalid input. Please enter a numerical quantity.")
except Exception as e:
    print("An unexpected error occurred with quantity input: " + str(e))

def sell_products(products_data):
    """
    Handles the process of selling one or more products to a customer.
    Updates product_data in memory and returns it along with bill details.
    """
    if not products_data:
        print("Inventory is empty. Cannot sell products.")
    return products_data, None

customer_name = input("Enter Customer Name: ")

sold_items_for_bill = []    current_sale_subtotal = 0.0
apply_clearance_discount = False # Flag to check if any item goes out of stock

while True:
    display_products(products_data)    product_id =
    get_valid_product_id(products_data, "Enter Product ID to sell (or 0 to
    finish current sale): ", allow_zero=True)

    if product_id == 0:    if
    not sold_items_for_bill:
        print("No products were selected for this sale.")
    return products_data, None    break

```

```

    product_details = products_data[product_id]
    product_name = product_details[0]
    current_stock = product_details[2]    cost_price
    = product_details[3]    selling_price_per_unit =
    cost_price * 2.0    print("Selected: " +
    product_name + " (Stock: " + str(current_stock) +
    ", Selling Price/unit: Rs. " + ("%0.2f" %
    selling_price_per_unit) + ")")
    requested_quantity = get_valid_quantity("Enter
    quantity to sell: ")

    free_items = requested_quantity // 3
    total_quantity_to_deduct_from_stock = requested_quantity + free_items

    if total_quantity_to_deduct_from_stock > current_stock:
        print("Error: Not enough stock for " + product_name + ".")    print("Available:
    " + str(current_stock) + ", Required (incl. free): " +
    str(total_quantity_to_deduct_from_stock) + ". Please try a smaller quantity.")
    continue

    products_data[product_id][2] -= total_quantity_to_deduct_from_stock    if
    products_data[product_id][2] == 0: # Check if stock became 0
    apply_clearance_discount = True    print("INFO: Product " + product_name + " is
    now out of stock. Clearance discount
    may apply to the bill.")

    item_sale_total = requested_quantity * selling_price_per_unit
    current_sale_subtotal += item_sale_total

    sold_items_for_bill.append({
        'id': product_id,

```

```

        'name': product_name,
        'quantity': requested_quantity,
        'price_per_unit': selling_price_per_unit,
        'total': item_sale_total,
        'free_items_received': free_items,
        'total_deducted_stock': total_quantity_to_deduct_from_stock
    })

    print(str(requested_quantity) + " " + product_name + "(s) added to bill.")
    if free_items > 0:
        print("BONUS: Customer receives " + str(free_items) + " " + product_name + "(s)
for free!")

    another_item_choice = input("Sell another product in this transaction? (yes/no): ")
    if not (another_item_choice == "yes" or another_item_choice == "YES" or \
another_item_choice == "y" or another_item_choice == "Y"):        break

    if not sold_items_for_bill:
        return products_data, None

# Sales Invoice Logic
discount_amount = 0.0    if
apply_clearance_discount:
    discount_amount = current_sale_subtotal * 0.10
    print("INFO: Applying 10% clearance discount to the subtotal.")

    amount_after_discount = current_sale_subtotal - discount_amount    vat_amount =
amount_after_discount * 0.13 # VAT is 13% of the (Subtotal - Discount)
grand_total_payable = amount_after_discount + vat_amount

    bill_details_for_writing = {
        "customer_name": customer_name,

```

```

    "items_list": sold_items_for_bill,
    "subtotal": current_sale_subtotal,
    "discount_amount": discount_amount,
    "vat_amount": vat_amount,
    "grand_total": grand_total_payable,
}

print("\n--- Sale Transaction Summary ---")
for item in sold_items_for_bill:
    print(item['name'] + " - Qty Paid: " + str(item['quantity']) + ", Free: " +
    str(item['free_items_received']) + ", Total Deducted: " + str(item['total_deducted_stock']))

return products_data, bill_details_for_writing

def purchase_products(products_data):
    """
    Handles purchasing products: restocking existing or adding new ones.
    Updates product_data in memory and returns it along with bill details.
    """
    print("\n--- Purchase from Manufacturer/Supplier ---")

    purchased_items_for_bill = []
    current_purchase_total_cost = 0.0

    while True:
        print("\nPurchase Options:")
        print("1. Restock an existing product")
        print("2. Add a completely new product")
        print("3. Finish current purchase transaction")

```

```

    action_choice_str = input("Enter your choice (1-3): ")
try:
    action_choice = int(action_choice_str)
except ValueError:
    print("Invalid choice. Please enter a number (1, 2, or 3).")
continue

    if action_choice == 1:
        if not products_data:
            print("No products exist to restock. Please add a
new product first (Option 2).")
            continue

        display_products(products_data)
        product_id =
get_valid_product_id(products_data, "Enter Product ID of existing
product to restock: ")

        product_details = products_data[product_id]
product_name = product_details[0]
cost_price
= product_details[3]

        print("Selected for restock: " + product_name + " (Current Stock: " +
str(product_details[2]) + ", Cost Price/unit: Rs. " + ("%0.2f" % cost_price) + ")")
        quantity_to_add = get_valid_quantity("Enter quantity to add to stock: ")

        products_data[product_id][2] += quantity_to_add
item_total_purchase_cost = quantity_to_add * cost_price
current_purchase_total_cost += item_total_purchase_cost

    purchased_items_for_bill.append({
        'id': product_id,
        'name': product_name,
        'brand': product_details[1],
        'quantity': quantity_to_add,

```

```

        'price_per_unit': cost_price,
        'total': item_total_purchase_cost
    })
    print(str(quantity_to_add) + " " + product_name + "(s) restocked. New stock: " +
    str(products_data[product_id][2]))

elif action_choice == 2:
    print("\n--- Add New Product to Inventory ---")
    new_product_name = input("Enter new product's Name: ")
    new_product_brand = input("Enter new product's Brand: ")
    new_product_quantity = get_valid_quantity("Enter initial quantity for this new
    product: ")

    new_product_cost_price = 0.0
    while True:
        try:
            new_product_cost_price_str = input("Enter cost price per unit for the new
            product: ")
            new_product_cost_price = float(new_product_cost_price_str)
            if new_product_cost_price <= 0:
                print("Cost price must be a positive number.")
            else:
                break
        except ValueError:
            print("Invalid price format. Please enter a number (e.g., 150.75).")

    new_product_origin = input("Enter new product's Country of Origin: ")

    new_id = 0
    if products_data:
        max_existing_id = 0
    for k_id in products_data.keys():

```

```

        if int(k_id) > int(max_existing_id): # Ensure comparison as int
max_existing_id = k_id          new_id = int(max_existing_id) + 1
else:

```

```

    new_id = 1

```

```

        products_data[new_id] = [new_product_name, new_product_brand,
new_product_quantity, new_product_cost_price, new_product_origin]
item_total_purchase_cost = new_product_quantity * new_product_cost_price
current_purchase_total_cost += item_total_purchase_cost

```

```

        purchased_items_for_bill.append({
            'id': new_id,
            'name': new_product_name,
            'brand': new_product_brand,
            'quantity': new_product_quantity,
            'price_per_unit': new_product_cost_price,
            'total': item_total_purchase_cost
        })
        print("New product " + new_product_name + " (ID: " + str(new_id) + ") added with
stock " + str(new_product_quantity) + ".")

```

```

        elif action_choice == 3:          if not purchased_items_for_bill:
print("No products were restocked or added in this transaction.")
return products_data, None          break          else:
        print("Invalid choice. Please enter 1, 2, or 3.")

```

```

        if not purchased_items_for_bill:
return products_data, None

```

```

        # Purchase Invoice Logic          discount_amount = 0.0 # No discount for purchases
vat_amount = 0.0          # No VAT for purchases          grand_total_paid_by_store =
current_purchase_total_cost # Grand total is just the

```

subtotal

```
bill_details_for_writing = {
    "customer_name": "Store Purchase (Self/Supplier)",
    "items_list": purchased_items_for_bill,
    "subtotal": current_purchase_total_cost,
    "discount_amount": discount_amount, # Will be 0
    "vat_amount": vat_amount,          # Will be 0
    "grand_total": grand_total_paid_by_store,
}
```

```
return products_data, bill_details_for_writing
```

FOR MAIN.py #

main.py import

read import write

import operations

Global constants for company details and filenames

COMPANY_NAME = "TopskinCare Wholesale"

COMPANY_ADDRESS = "Madhyapur, Thimi, Bhaktapur"

PRODUCTS_FILENAME = "Products.txt"

BILL_COUNTER_FILENAME = "bill_counter.txt"

def display_welcome_message():

 """Prints the welcome message for the system."""

print("\n" * 2) print("\t\t\t\t\t" + COMPANY_NAME)

 print("\n")

 print("\t\t\t\t\t" + COMPANY_ADDRESS)

 print("\n")

def main_program_loop():

SUPREM THAPA

""Runs the main interactive loop for the wholesale system.""

```
display_welcome_message()
```

```
products_inventory = read.load_products(PRODUCTS_FILENAME)
```

```
system_active = True
```

```
while system_active:
```

```
    print("\n===== Main Menu =====")
```

```
    print("1. Sell Product(s) to Customer")    print("2.
```

```
Purchase Product(s) (Restock/Add New)")    print("3.
```

```
Display All Available Products")    print("4. Exit
```

```
System")
```

```
    print("=====")
```

```
    user_choice_str = input("Enter your choice (1-4): ")
```

```
try:
```

```
    main_menu_choice = int(user_choice_str)
```

```
except ValueError:
```

```
    print("Error: Invalid input! Please enter a number between 1 and 4.")
```

```
continue    except Exception as e:
```

```
    print("An unexpected error occurred with your input: " + str(e))
```

```
continue
```

```
    if main_menu_choice == 1: # Sell Products
```

```
        updated_inventory_after_sale, sale_bill_details =
operations.sell_products(products_inventory)
```

```
        if
```

```
sale_bill_details:
```

```
            products_inventory = updated_inventory_after_sale
```

```

        current_bill_num = read.read_bill_counter(BILL_COUNTER_FILENAME)
new_bill_num = current_bill_num + 1

        unique_bill_id = write.generate_unique_bill_id(new_bill_num)

        write.print_and_save_bill(
unique_bill_id,
sale_bill_details["customer_name"],
sale_bill_details["items_list"],
sale_bill_details["subtotal"],
sale_bill_details["discount_amount"],
sale_bill_details["vat_amount"],
sale_bill_details["grand_total"],
        "SALE",
        COMPANY_NAME,
        COMPANY_ADDRESS
    )
    write.save_products(PRODUCTS_FILENAME, products_inventory)
write.write_bill_counter(new_bill_num, BILL_COUNTER_FILENAME)      else:
    print("Sale operation did not complete or no items were sold. No bill
generated.")

elif main_menu_choice == 2: # Purchase Products
    updated_inventory_after_purchase,    purchase_bill_details    =
operations.purchase_products(products_inventory)

    products_inventory = updated_inventory_after_purchase # Persist any changes
even if no bill

    if purchase_bill_details:
        current_bill_num = read.read_bill_counter(BILL_COUNTER_FILENAME)
new_bill_num = current_bill_num + 1                unique_bill_id =
write.generate_unique_bill_id(new_bill_num)

```

```

        write.print_and_save_bill(
            unique_bill_id,
purchase_bill_details["customer_name"],
purchase_bill_details["items_list"],
purchase_bill_details["subtotal"],
purchase_bill_details["discount_amount"], # Will be 0
purchase_bill_details["vat_amount"],      # Will be 0
purchase_bill_details["grand_total"],
            "PURCHASE",
            COMPANY_NAME,
            COMPANY_ADDRESS
        )
        write.write_bill_counter(new_bill_num, BILL_COUNTER_FILENAME)
    else:
        print("Purchase operation did not complete with billable items or was
cancelled.")

        write.save_products(PRODUCTS_FILENAME, products_inventory) # Save after
any purchase op

    elif main_menu_choice == 3: # Display Products
operations.display_products(products_inventory)

        elif main_menu_choice == 4: # Exit            print("\nThank you for
using the " + COMPANY_NAME + " system.")            print("Exiting now...")
system_active = False

    else:
        print("Error: Invalid choice. Please select an option from 1 to 4.")

```

```
if __name__ == "__main__":  
    main_program_loop()
```